OPERATING SYSTEMS NOTES

Theory explained step by step

Author: SamuMGH04

WELCOME

Thanks for choosing these notes. This project is created with the aim of making people understand what's behind an Operating System and everything related to it.

LICENSE

These works are protected and distributed under the license
Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International

- **Attribution:** You must mention the original creator
- **Non Commercial:** You can't sell this material
- **No Derivatives:** You can't modify or change the text.

SISTEMI OPERATIVI: LEZIONE 3

Indice

1 Pianificazione della CPU e Multiprogrammazione **10**
1.1 Introduzione 10
1.2 Sistemi a singolo core e multiprogrammazione 10
1.3 Idea chiave 10
1.4 Osservazioni 10

2 Burst di CPU e Burst di I/O **11**
2.1 Definizione dei burst 11
2.2 Ciclo di esecuzione di un processo 11
2.3 Distribuzione dei burst 11
2.4 Idea chiave 11
2.5 Osservazioni 12

3 Pianificazione della CPU e Coda dei Pronti **13**
3.1 Coda dei pronti 13
3.2 Ruolo dello scheduler della CPU 13
3.3 Strutture della coda dei pronti 13
3.4 Idea chiave 13
3.5 Osservazioni 14

4 Decisioni di Pianificazione della CPU **14**
4.1 Introduzione 14
4.2 Momenti in cui avviene la pianificazione 14
4.3 Scheduling non-preemptive e preemptive 15
4.4 Idea chiave 15
4.5 Osservazioni 15

5 Il Dispatcher **16**
5.1 Introduzione 16
5.2 Principali compiti del dispatcher 16
5.3 Efficienza e latenza di dispatch 16
5.4 Idea chiave 16
5.5 Osservazioni 17

6 Monitoraggio dei Context Switch in Linux **17**
6.1 Introduzione . . . 17
6.2 Monitoraggio a livello di sistema con `vmstat` . . . 17
6.3 Monitoraggio a livello di processo con /proc/[pid]/status . . . 18
6.4 Idea chiave . . . 18
6.5 Osservazioni . . . 18

7 Criteri di Pianificazione della CPU **19**
7.1 Introduzione . . . 19
7.2 Principali metriche di prestazione . . . 19
7.3 Obiettivi della pianificazione . . . 19
7.4 Idea chiave . . . 20
7.5 Osservazioni . . . 20

8 Schedulazione First-Come, First-Served (FCFS) **20**
8.1 Introduzione . . . 20
8.2 Esempio pratico . . . 20
8.3 Analisi del problema . . . 21
8.4 Osservazioni . . . 21

9 Schedulazione Shortest Job First (SJF) **22**
9.1 Introduzione . . . 22
9.2 Esempio pratico . . . 22
9.3 Idea chiave . . . 22
9.4 Osservazioni . . . 23

10 Approssimazione dello Shortest Job First (SJF) **23**
10.1 Introduzione . . . 23
10.2 Predizione dei burst futuri tramite media esponenziale . . . 23
10.3 Significato del parametro α . . . 23
10.4 Idea chiave . . . 24
10.5 Osservazioni . . . 24

11 SJF Preemptive vs Non-Preemptive **24**
11.1 Introduzione . . . 24
11.2 SJF Non-Preemptive . . . 24
11.3 SJF Preemptive (Shortest Remaining Time First, SRTF) . . . 24
11.4 Esempio di confronto . . . 25
11.5 Idea chiave . . . 25
11.6 Osservazioni . . . 25

12 Schedulazione Round Robin (RR) **26**
12.1 Introduzione . . . 26
12.2 Dipendenza dalle dimensioni del quantum . . . 26
12.3 Esempi numerici . . . 26
12.3.1 Un processo di 10 unità . . . 26
12.3.2 Tre processi di 10 unità ciascuno . . . 26
12.4 Idea chiave . . . 26
12.5 Osservazioni . . . 27

13 Schedulazione a Priorità (Priority Scheduling) 27
13.1 Introduzione . . . 27
13.2 Esempio di priorità basata sul burst . . . 27
13.3 Problema della starvation (invecchiamento) . . . 28
13.4 Idea chiave . . . 28
13.5 Osservazioni . . . 28

14 Risoluzione della Starvation: Aging e Priority + Round Robin 29
14.1 Introduzione . . . 29
14.2 Esempio pratico . . . 29
14.3 Aging . . . 29
14.4 Schedulazione ibrida: Priority + Round Robin . . . 29
14.5 Idea chiave . . . 30
14.6 Osservazioni . . . 30

15 Schedulazione a Code Multilivello (Multilevel Queue Scheduling) 30
15.1 Introduzione . . . 30
15.2 Struttura del sistema . . . 30
15.3 Vantaggi principali . . . 30
15.4 Priorità statica . . . 31
15.5 Idea chiave . . . 31
15.6 Classificazione dei processi e politiche per coda . . . 31
15.7 Idea chiave . . . 31
15.8 Osservazioni . . . 32

16 Schedulazione a Code Multilivello con Feedback (Multilevel Feedback Queue, MLFQ) 32
16.1 Introduzione . . . 32
16.2 Meccanismo di feedback . . . 32
16.3 Definizione della MLFQ . . . 32
16.4 Idea chiave . . . 33
16.5 Osservazioni . . . 33

17 Schedulazione dei Thread: User-Level vs Kernel-Level 34
17.1 Introduzione . . . 34
17.2 Mappatura dei thread . . . 34
17.3 Scope di schedulazione . . . 34
17.4 Idea chiave . . . 35
17.5 Osservazioni . . . 35

18 Pthreads: Contention Scope 36
18.1 Introduzione . . . 36
18.2 Policy disponibili . . . 36
18.3 Comportamento nei sistemi Many-to-Many . . . 36
18.4 Funzioni API per gestire lo scope . . . 36
18.5 Idea chiave . . . 36
18.6 Osservazioni . . . 37

19 Schedulazione su Multiprocessore **37**
19.1 Introduzione . . . 37
19.2 Significato moderno di "multiprocessore" . . . 37
19.3 Multiprocessing asimmetrico (Asymmetric Multiprocessing) . . . 37
19.4 Multiprocessing simmetrico (Symmetric Multiprocessing, SMP) . . . 38
19.5 Multicore su singolo chip . . . 38
19.6 Sfide nello scheduling su multiprocessore . . . 38
19.7 Idea chiave . . . 38
19.8 Osservazioni . . . 38

20 Chip Multithreading (CMT) **39**
20.1 Introduzione . . . 39
20.2 Tipi di Multithreading . . . 39
20.2.1 Coarse-Grained Multithreading (Multithreading Grossolano) . . . 39
20.2.2 Fine-Grained (Interleaved) Multithreading (Multithreading Fine/Interlacciato) 39
20.3 Idea chiave . . . 39
20.4 Osservazioni . . . 40

21 Schedulazione a Due Livelli nei Sistemi Multicore Multithreaded **40**
21.1 Introduzione . . . 40
21.2 I due livelli di schedulazione . . . 40
21.3 Esempi di implementazione . . . 40
21.4 Idea chiave . . . 41
21.5 Osservazioni . . . 41

22 Bilanciamento del Carico nei Sistemi SMP (Load Balancing in SMP Systems) **42**
22.1 Introduzione . . . 42
22.2 Strategie di migrazione dei processi . . . 42
22.3 Idea chiave . . . 42
22.4 Osservazioni . . . 43

23 Processor Affinity nei Sistemi SMP **43**
23.1 Introduzione . . . 43
23.2 Vantaggi dell'affinità . . . 43
23.3 Tipi di Processor Affinity . . . 43
23.4 Idea chiave . . . 44
23.5 Osservazioni . . . 44

24 Schedulazione della CPU in Tempo Reale (Real-Time CPU Scheduling) **45**
24.1 Introduzione . . . 45
24.2 Tipologie di sistemi real-time . . . 45
24.3 Idea chiave . . . 45
24.4 Osservazioni . . . 46

25 Event Latency nei Sistemi Real-Time **46**
25.1 Introduzione . . . 46
25.2 Definizione di Event Latency . . . 46
25.3 Requisiti di latenza in diversi contesti . . . 46

25.4 Idea chiave 46
25.5 Osservazioni 47

26 Latency nei Sistemi Real-Time **47**
26.1 Tipi principali di latenza 47
26.2 Interrupt Latency 47
26.3 Idea chiave 47
26.4 Osservazioni 48

27 Interrupt e Dispatch Latency nei Sistemi Real-Time **48**
27.1 Introduzione 48
27.2 Riduzione della Interrupt Latency 48
27.3 Dispatch Latency 48
27.4 Misurazioni nei sistemi hard real-time 49
27.5 Idea chiave 49
27.6 Osservazioni 49

28 Schedulazione Hard Real-Time **50**
28.1 Introduzione 50
28.2 Politica di Scheduling 50
28.3 Caratteristiche dei Task Periodici 50
28.4 Admission Control 50
28.5 Idea chiave 50
28.6 Osservazioni 51

29 Schedulazione Rate-Monotonic (RMS) **51**
29.1 Introduzione 51
29.2 Principio di funzionamento 51
29.3 Assunzioni RMS 51
29.4 Idea chiave 52
29.5 Osservazioni 52

30 Verifica di Fattibilità con Due Task Periodici **53**
30.1 Introduzione 53
30.2 Dati del problema 53
30.3 Calcolo dell'utilizzo della CPU 53
30.4 Assegnazione di priorità non ottimale 53
30.5 Esecuzione prevista 53
30.6 Risultato 53
30.7 Spiegazione 54
30.8 Idea chiave 54

31 Verifica di Fattibilità con Priorità Corretta per Due Task Periodici **55**
31.1 Introduzione 55
31.2 Assegnazione di priorità corretta 55
31.3 Esecuzione prevista con preemption 55
31.4 Risultato 55
31.5 Spiegazione dettagliata 56
31.6 Idea chiave 56

31.7 Osservazioni 56

32 Limitazioni della Schedulazione a Priorità Basata sulla Frequenza 57
32.1 Introduzione 57
32.2 Principio di ottimalità 57
32.3 Esempio pratico 57
32.4 Calcolo dell'utilizzo della CPU 57
32.5 Analisi 57
32.6 Conclusione 57
32.7 Idea chiave 58

33 Limite di Utilizzo della CPU nella Schedulazione a Priorità Basata sulla Frequenza 58
33.1 Introduzione 58
33.2 Limite massimo di utilizzo per N task 58
33.3 Esempi pratici 58
33.4 Spiegazione 58
33.5 Idea chiave 59
33.6 Osservazioni 59

34 Schedulazione Earliest-Deadline-First (EDF) 60
34.1 Introduzione 60
34.2 Esempio pratico 60
34.3 Esecuzione con EDF 60
34.4 Vantaggi di EDF 60
34.5 Limitazioni pratiche 61
34.6 Idea chiave 61

35 Schedulazione a Quote Proporzionali (Proportional Share Scheduling) 62
35.1 Introduzione 62
35.2 Principio di funzionamento 62
35.3 Esempio pratico 62
35.4 Gestione tramite Admission Control 62
35.5 Idea chiave 62
35.6 Osservazioni 63

36 Esempio di Valutazione Analitica degli Algoritmi di Schedulazione 63
36.1 Introduzione 63
36.2 Workload considerato 63
36.3 Confronto tra algoritmi 64
36.4 Conclusione 64
36.5 Idea chiave 64

37 Valutazione della Schedulazione tramite Simulazione 65
37.1 Introduzione 65
37.2 Principi generali 65
37.3 Metodi di simulazione 65
37.4 Vantaggi della simulazione 65
37.5 Idea chiave 65

38 Il Problema della Sezione Critica **66**
38.1 Introduzione 66
38.2 Sezione critica 66
38.3 Protocollo per la cooperazione 66
38.4 Struttura di un processo cooperante 66
38.5 Idea chiave 66

39 Requisiti per le Soluzioni della Sezione Critica **67**
39.1 Introduzione 67
39.2 1. Mutua esclusione (Mutual Exclusion) 67
39.3 2. Progress (Progresso) 67
39.4 3. Attesa limitata (Bounded Waiting) 67
39.5 Idea chiave 68

40 Sezioni Critiche e Progettazione del Kernel **68**
40.1 Introduzione 68
40.2 Single-Core Systems 68
40.3 Multiprocessor Systems 68
40.4 Strategie del Kernel 69
40.5 Preferenze pratiche 69
40.6 Idea chiave 69

41 Soluzione di Peterson per il Problema della Sezione Critica **70**
41.1 Introduzione 70
41.2 Variabili condivise 70
41.3 Idea principale 70
41.4 Funzionamento passo passo 70
41.5 Codice in pseudolinguaggio 71
41.6 Spiegazione dettagliata del funzionamento 71
41.7 Idea chiave 71

42 Perché la Soluzione di Peterson Funziona **72**
42.1 Introduzione 72
42.2 Variabili condivise 72
42.3 Mutua esclusione 72
42.4 Progresso 72
42.5 Attesa limitata (Bounded Waiting) 72
42.6 Idea chiave 73

43 Mutex Locks **73**
43.1 Introduzione 73
43.2 Funzionamento base 73
43.3 Problema della Busy Waiting 73
43.4 Schema di utilizzo di un mutex 74
43.5 Idea chiave 74

44 Semafori **75**
44.1 Introduzione 75
44.2 Definizione 75
44.3 Proprietà delle operazioni 75
44.4 Importanza dell'atomicità 75
44.5 Idea chiave 75
44.6 Schema di utilizzo di un semaforo 76

45 Semafori Contatori e Binari **76**
45.1 Introduzione 76
45.2 Semafori binari 76
45.3 Semafori contatori 76
45.4 Idea chiave 77

46 Semafori per la Sincronizzazione dei Processi **77**
46.1 Introduzione 77
46.2 Meccanismo base 77
46.3 Codice in pseudolinguaggio 77
46.4 Spiegazione dettagliata 78
46.5 Idea chiave 78

47 Evitare la Busy Waiting con i Semafori **79**
47.1 Introduzione 79
47.2 Blocco del processo 79
47.3 signal() e wakeup() 79
47.4 Schema concettuale 79
47.5 Idea chiave 80

48 Struttura e Operazioni dei Semafori **80**
48.1 Struttura di un Semaforo 80
48.2 Operazione wait() 80
48.3 Operazione signal() 80
48.4 Schema concettuale 81
48.5 Idea chiave 81

49 Comportamento della Coda dei Semafori **82**
49.1 Introduzione 82
49.2 Valore del semaforo 82
49.3 Struttura del semaforo 82
49.4 Politica della coda 82
49.5 Funzionamento 82
49.6 Idea chiave 82

50 Limiti dei Semafori **83**
50.1 Introduzione 83
50.2 Problemi comuni 83
50.3 Esempio di errore 83
50.4 Idea chiave 83

51 Monitor **84**
51.1 Introduzione 84
51.2 Caratteristiche principali 84
51.3 Encapsulamento e sicurezza 84
51.4 Schema di Sintassi Tipica di un Monitor 85
51.5 Note sul funzionamento 85
51.6 Idea chiave 85

52 Variabili di Condizione nei Monitor **86**
52.1 Introduzione 86
52.2 Definizione 86
52.3 Funzionamento di wait() e signal() 86
52.4 Idea chiave 86

53 Comportamento di `signal()` con le Variabili di Condizione **87**
53.1 Introduzione 87
53.2 Comportamento base di signal() 87
53.3 Caso con processi sospesi 87
53.4 Politiche di risveglio 87
53.5 Idea chiave 87

54 Implementazione dei Monitor tramite Semafori **88**
54.1 Introduzione 88
54.2 Semafori utilizzati 88
54.3 Funzionamento della politica signal-and-wait 88
54.4 Schema concettuale di implementazione 89
54.5 Idea chiave 89

55 Implementazione delle Variabili di Condizione tramite Semafori **90**
55.1 Introduzione 90
55.2 Struttura per ogni variabile di condizione 90
55.3 Funzionamento di `x.wait()` 90
55.4 Funzionamento di `x.signal()` 90
55.5 Schema concettuale di pseudocodice 91
55.6 Idea chiave 91

56 Ripresa dei Processi all'interno di un Monitor **92**
56.1 Introduzione 92
56.2 Strategie di selezione del processo 92
56.3 Esempio concettuale 92
56.4 Idea chiave 92

1 Pianificazione della CPU e Multiprogrammazione

1.1 Introduzione

Nel contesto dei sistemi operativi, la gestione efficiente della CPU è fondamentale per garantire alte prestazioni. La **pianificazione della CPU** (*CPU scheduling*) è la strategia mediante la quale il sistema operativo decide quale processo o thread deve essere eseguito dalla CPU in un dato momento. L'obiettivo principale della pianificazione della CPU è *massimizzare l'utilizzo della CPU*, minimizzando i tempi di inattività e garantendo un alto *throughput* del sistema.

1.2 Sistemi a singolo core e multiprogrammazione

- **Sistema a singolo core**: in un sistema con un solo core, la CPU può eseguire un solo processo alla volta. Anche se il sistema può gestire più processi in memoria, la CPU esegue sequenzialmente un processo alla volta. Ciò può portare a periodi in cui la CPU è inattiva, ad esempio quando il processo in esecuzione è in attesa di I/O.

- **Multiprogrammazione**: consiste nel mantenere in memoria più processi contemporaneamente. Grazie alla multiprogrammazione, quando un processo entra in stato di attesa (ad esempio per operazioni di input/output), la CPU può immediatamente passare a un altro processo pronto, riducendo i tempi morti e migliorando l'utilizzo della CPU. In altre parole, la multiprogrammazione permette alla CPU di avere sempre un processo pronto all'esecuzione, aumentando l'efficienza complessiva del sistema.

- **Sistema multicore**: estende il concetto di multiprogrammazione su più core fisici. In un sistema multicore, più processi possono essere eseguiti in parallelo, ognuno su un core diverso. Questo aumenta significativamente il throughput e riduce i tempi di risposta rispetto a un sistema a singolo core.

1.3 Idea chiave

Il principio fondamentale della multiprogrammazione è che **massimizza l'utilizzo della CPU e l'efficienza del sistema** assicurando che ci sia sempre almeno un processo in esecuzione o pronto per essere eseguito. In questo modo, la CPU rimane occupata il più possibile, evitando inutili periodi di inattività.

1.4 Osservazioni

- La multiprogrammazione non aumenta la velocità di esecuzione di un singolo processo, ma migliora il *throughput* totale del sistema.

- La pianificazione efficace della CPU è fondamentale soprattutto nei sistemi a singolo core, dove la scelta del processo successivo determina direttamente l'efficienza del processore.

- Nei sistemi multicore, il sistema operativo deve gestire anche l'assegnazione dei processi ai core, oltre alla scelta dei processi pronti.

2 Burst di CPU e Burst di I/O

2.1 Definizione dei burst

Nei sistemi operativi, l'esecuzione di un processo può essere suddivisa in fasi distinte chiamate **burst**. Esistono due tipi principali:

- **Burst di CPU** (*CPU burst*): è una sequenza di operazioni eseguite direttamente dalla CPU, durante la quale il processo utilizza intensamente il processore senza attendere input/output. Durante un burst di CPU, il processo esegue calcoli, manipolazioni di dati e altre istruzioni di elaborazione pura.
- **Burst di I/O** (*I/O burst*): è una sequenza di operazioni di input/output, durante la quale il processo interagisce con dispositivi esterni come dischi, tastiere, stampanti o reti. Durante un burst di I/O, la CPU non è attivamente impegnata dal processo, permettendo al sistema operativo di assegnarla ad altri processi pronti.

2.2 Ciclo di esecuzione di un processo

Un processo tipicamente alterna tra burst di CPU e burst di I/O secondo un ciclo ricorrente:

1. Il processo inizia con un *CPU burst*, durante il quale esegue calcoli e operazioni logiche.
2. Al termine del burst di CPU, il processo può richiedere un'operazione di I/O, entrando in uno stato di attesa per il completamento del burst di I/O.
3. Terminato il burst di I/O, il processo ritorna nello stato pronto e può essere selezionato per un nuovo burst di CPU.
4. Questo ciclo continua fino all'ultimo burst di CPU, che termina con una richiesta di terminazione del processo.

2.3 Distribuzione dei burst

I processi possono essere classificati in base alla durata e frequenza dei loro burst di CPU:

- **Processi I/O-bound**: caratterizzati da molti *CPU burst* brevi e frequenti. La loro esecuzione è limitata principalmente dalle operazioni di I/O, mentre la CPU è spesso inattiva in attesa dei dispositivi.
- **Processi CPU-bound**: caratterizzati da pochi *CPU burst* lunghi. Questi processi utilizzano intensamente la CPU e richiedono meno operazioni di I/O.

2.4 Idea chiave

La lunghezza dei burst di CPU non è uniforme ma segue tipicamente una **distribuzione esponenziale o iper-esponenziale**. Questo significa che la maggior parte dei burst di CPU è breve, ma occasionalmente possono verificarsi burst molto lunghi. La conoscenza statistica di queste distribuzioni è fondamentale per progettare strategie di *CPU scheduling* efficienti, poiché algoritmi diversi (come SJF o Round Robin) rispondono in maniera diversa a burst di lunghezza variabile.

2.5 Osservazioni

- La distinzione tra processi I/O-bound e CPU-bound guida la scelta dell'algoritmo di schedulazione ottimale: ad esempio, un algoritmo che favorisce processi brevi migliora il tempo di risposta dei processi I/O-bound.
- La natura esponenziale dei burst implica che non è possibile prevedere con certezza la durata di un singolo burst, ma è possibile stimare statisticamente le medie e i tempi di attesa.
- I sistemi operativi moderni utilizzano queste informazioni per implementare scheduler *adaptive*, capaci di bilanciare equamente i processi I/O-bound e CPU-bound.

3 Pianificazione della CPU e Coda dei Pronti

3.1 Coda dei pronti

Nel contesto della pianificazione della CPU, tutti i processi che sono pronti per essere eseguiti dalla CPU vengono mantenuti in una struttura dati chiamata **coda dei pronti** (*ready queue*). La coda dei pronti è fondamentale perché permette al sistema operativo di tenere traccia di tutti i processi che sono in attesa di ottenere la CPU, garantendo che nessun processo pronto venga ignorato.

Ogni processo nella coda dei pronti è rappresentato da un **Process Control Block** (PCB), che contiene informazioni essenziali sul processo, come lo stato, il contatore del programma (Program Counter), i registri della CPU, e altre informazioni di scheduling.

3.2 Ruolo dello scheduler della CPU

Il **scheduler della CPU** è il componente del sistema operativo responsabile della selezione del prossimo processo da eseguire tra quelli presenti nella coda dei pronti. La scelta dipende dall'algoritmo di scheduling adottato (ad esempio First-Come, First-Served, Shortest Job First, Round Robin, ecc.) e dalle caratteristiche dei processi.

3.3 Strutture della coda dei pronti

La coda dei pronti può essere organizzata in diverse strutture dati a seconda della strategia di schedulazione:

- **FIFO (First-In-First-Out)**: i processi vengono eseguiti nell'ordine in cui arrivano nella coda. È semplice da implementare, ma può penalizzare processi brevi se un processo lungo arriva prima.
- **Coda con priorità**: i processi vengono selezionati in base alla loro priorità. Un processo con priorità più alta viene eseguito prima di processi a priorità più bassa. La priorità può essere statica (assegnata una volta) o dinamica (cambiata durante l'esecuzione).
- **Albero (Tree)**: la coda può essere implementata come struttura ad albero, ad esempio un heap, per consentire l'accesso efficiente al processo con priorità più alta o al processo più breve.
- **Lista collegata non ordinata**: semplice da implementare, ma richiede una scansione completa della lista per selezionare il processo successivo in base all'algoritmo di scheduling.

3.4 Idea chiave

La coda dei pronti contiene i PCB dei processi pronti e può essere organizzata in modi diversi a seconda della strategia di pianificazione della CPU. La scelta della struttura dati influisce direttamente sull'efficienza dello scheduler e sulle prestazioni complessive del sistema operativo.

3.5 Osservazioni

- La gestione efficiente della coda dei pronti è cruciale nei sistemi con molti processi, poiché l'accesso e l'aggiornamento dei PCB devono essere rapidi per evitare colli di bottiglia.
- Algoritmi diversi richiedono strutture diverse: ad esempio, un algoritmo basato sulla priorità beneficia di una coda con accesso rapido al massimo, mentre un algoritmo FIFO può utilizzare una semplice lista concatenata.
- Lo scheduler della CPU può anche combinare più criteri, ad esempio una coda FIFO con priorità dinamica, per bilanciare equità e performance.

4 Decisioni di Pianificazione della CPU

4.1 Introduzione

La **pianificazione della CPU** consiste nel decidere quale processo tra quelli pronti deve essere eseguito dalla CPU in un dato momento. Questa decisione non avviene casualmente, ma in momenti ben definiti durante il ciclo di vita di un processo. Comprendere quando e come avviene la pianificazione è fondamentale per scegliere l'algoritmo di scheduling più adatto e per garantire efficienza e reattività del sistema operativo.

4.2 Momenti in cui avviene la pianificazione

La CPU viene pianificata nei seguenti quattro casi principali:

1. **Processo da eseguire a in attesa** (*running* $\rightarrow$ *waiting*): avviene quando un processo in esecuzione richiede un'operazione di I/O o chiama una funzione di attesa come `wait()`. In questo caso, la CPU viene liberata e deve essere assegnata a un altro processo pronto.
2. **Processo da eseguire a pronto** (*running* $\rightarrow$ *ready*): si verifica, ad esempio, quando un interrupt interrompe il processo corrente. Il processo non è più in esecuzione ma rimane pronto, in attesa che la CPU lo esegua nuovamente.
3. **Processo da attesa a pronto** (*waiting* $\rightarrow$ *ready*): avviene quando un processo che era in attesa di I/O completa la sua operazione. Il processo ritorna nello stato pronto e può essere selezionato dallo scheduler.
4. **Terminazione del processo**: quando un processo termina la sua esecuzione, libera la CPU, che può essere assegnata al prossimo processo pronto.

4.3 Scheduling non-preemptive e preemptive

Esistono due modalità principali di pianificazione della CPU, a seconda della possibilità di interrompere un processo in esecuzione:

- **Scheduling non-preemptive**: il processo in esecuzione non può essere interrotto dalla CPU finché non termina il suo burst di CPU o non richiede un'operazione di I/O (casi 1 e 4). Questo approccio richiede meno controllo da parte dello scheduler ma può risultare meno flessibile.
- **Scheduling preemptive**: il processo in esecuzione può essere interrotto in qualsiasi momento, non solo quando termina o entra in attesa, ma anche nei casi 2 e 3, cioè quando arriva un interrupt o un processo in attesa diventa pronto. Questo approccio permette una maggiore reattività del sistema e un bilanciamento più equo tra processi.

4.4 Idea chiave

Lo **scheduler non-preemptive** richiede meno complessità di controllo, ma può portare a un utilizzo subottimale della CPU e tempi di risposta più lunghi. Lo **scheduler preemptive**, invece, permette al sistema operativo di reagire rapidamente ai cambiamenti nello stato dei processi, migliorando la flessibilità e la reattività complessiva del sistema.

4.5 Osservazioni

- La scelta tra scheduling preemptive e non-preemptive dipende dagli obiettivi del sistema: sistemi interattivi beneficiano del preemptive, mentre sistemi batch possono tollerare il non-preemptive.
- Lo scheduling preemptive richiede strutture dati più sofisticate e meccanismi di interrupt ben progettati.
- In sistemi real-time, la preemption è spesso necessaria per garantire che processi critici rispettino le scadenze temporali.

5 Il Dispatcher

5.1 Introduzione

Nel ciclo di pianificazione della CPU, lo **scheduler a breve termine** seleziona quale processo deve essere eseguito successivamente. Tuttavia, la selezione da sola non è sufficiente: occorre un componente che effettivamente trasferisca il controllo della CPU al processo scelto. Questo ruolo è svolto dal **dispatcher**.

Il dispatcher è essenziale perché realizza il *passaggio effettivo* della CPU tra processi, garantendo che il processo selezionato inizi a eseguire correttamente e rapidamente.

5.2 Principali compiti del dispatcher

I compiti fondamentali del dispatcher includono:

1. **Switch di contesto tra processi** (*context switch*): salvare lo stato del processo corrente (registri, contatore del programma, altre informazioni di esecuzione) nel suo PCB e caricare lo stato del processo successivo. Questo passaggio assicura che ogni processo riprenda l'esecuzione esattamente da dove era stato interrotto.

2. **Cambio a modalità utente** (*switch to user mode*): la CPU passa dalla modalità kernel, in cui il dispatcher opera, alla modalità utente, in cui il processo può eseguire le proprie istruzioni. Questo è necessario per proteggere le risorse di sistema e garantire la sicurezza.

3. **Salto all'istruzione successiva del programma utente** (*jump to user program's next instruction*): il dispatcher deve avviare l'esecuzione del processo dall'istruzione corretta, come indicato dal contatore del programma del PCB.

5.3 Efficienza e latenza di dispatch

Il dispatcher deve essere estremamente veloce per ridurre l'overhead associato al cambio di contesto. Il tempo impiegato per completare il passaggio da un processo all'altro è chiamato **latenza di dispatch** (*dispatch latency*).

- Una bassa latenza di dispatch è cruciale per la performance dei sistemi, in particolare per quelli interattivi o con preemption frequente.

- La latenza include il tempo per salvare lo stato del processo corrente, caricare quello del nuovo processo e passare alla modalità utente.

5.4 Idea chiave

Il dispatcher realizza l'effettivo *handover* della CPU durante la pianificazione dei processi. Senza il dispatcher, la selezione dello scheduler non potrebbe tradursi in esecuzione reale, rendendo inefficace qualsiasi strategia di CPU scheduling.

5.5 Osservazioni

- Il dispatcher lavora in stretta collaborazione con lo scheduler a breve termine: lo scheduler decide *chi* esegue, il dispatcher decide *come* iniziare l'esecuzione.
- L'ottimizzazione della latenza di dispatch è particolarmente importante nei sistemi con scheduling preemptive, dove i processi vengono frequentemente interrotti.
- Meccanismi hardware come registri speciali o istruzioni dedicate possono ridurre il tempo necessario per un context switch, migliorando l'efficienza complessiva del dispatcher.

6 Monitoraggio dei Context Switch in Linux

6.1 Introduzione

Nei sistemi operativi moderni, il **context switch** è l'operazione mediante la quale il dispatcher salva lo stato di un processo e carica quello di un altro processo per continuare l'esecuzione. Monitorare i context switch è fondamentale per analizzare le prestazioni della CPU, comprendere l'overhead del sistema e ottimizzare lo scheduling dei processi.

In Linux, esistono strumenti sia a livello di sistema che a livello di processo per osservare i context switch.

6.2 Monitoraggio a livello di sistema con `vmstat`

Il comando `vmstat` permette di visualizzare statistiche sul sistema, inclusi i context switch per secondo. La sintassi base è:

```
vmstat 1
```

Dove l'intervallo di 1 secondo mostra le statistiche aggiornate ogni secondo.

- La prima riga di output mostra la media dei context switch dall'avvio del sistema.
- La seconda e terza riga mostrano i valori dei context switch rispettivamente per gli ultimi due intervalli di 1 secondo.

Esempio di output:

```
---- cpu ----
24
225
339
```

In questo esempio, la CPU ha effettuato in media 24 context switch dall'avvio, e negli ultimi due intervalli di 1 secondo rispettivamente 225 e 339 context switch.

6.3 Monitoraggio a livello di processo con /proc/[pid]/status

Linux fornisce anche informazioni dettagliate per ciascun processo attraverso il file `/proc/[pid]/status`. Qui è possibile monitorare il numero totale di context switch di un processo specifico, distinguendo tra due tipologie:

- **Voluntary switches (context switch volontari)**: si verificano quando un processo cede volontariamente la CPU, ad esempio perché attende il completamento di una risorsa di I/O.
- **Involuntary switches (context switch involontari)**: si verificano quando il processo viene preemptato dallo scheduler e la CPU viene assegnata a un altro processo, senza che il processo corrente abbia terminato il burst di CPU.

Esempio di output per il processo con PID 2166:

```
voluntary_ctxt_switches 150
nonvoluntary_ctxt_switches 8
```

In questo caso, il processo ha effettuato 150 context switch volontari e 8 involontari.

6.4 Idea chiave

Monitorare i context switch permette di comprendere l'efficienza dello scheduler e l'overhead associato al cambio di contesto. Un elevato numero di context switch involontari può indicare una CPU altamente contesa o una frequente preemption, mentre un numero basso di context switch volontari può segnalare processi CPU-bound che occupano la CPU per lunghi periodi.

6.5 Osservazioni

- L'analisi dei context switch aiuta a identificare colli di bottiglia e a ottimizzare gli algoritmi di scheduling.
- Strumenti come `vmstat` forniscono una visione globale del sistema, mentre i file in `/proc` permettono il monitoraggio dettagliato a livello di singolo processo.
- In sistemi multiprocessore, un elevato numero di context switch può influire sulle performance complessive, richiedendo un bilanciamento efficiente tra processi CPU-bound e I/O-bound.

7 Criteri di Pianificazione della CPU

7.1 Introduzione

Per valutare l'efficacia di un algoritmo di **CPU scheduling**, è necessario definire delle metriche di prestazione chiare. Queste metriche permettono di confrontare diverse strategie e determinare quale massimizza l'efficienza del sistema in relazione agli obiettivi desiderati. Gli obiettivi principali consistono nel mantenere la CPU il più possibile attiva, completare molti processi per unità di tempo e garantire tempi di risposta prevedibili e rapidi.

7.2 Principali metriche di prestazione

1. **Utilizzo della CPU (*CPU Utilization*)**: misura la percentuale di tempo in cui la CPU è effettivamente occupata nell'eseguire processi. L'obiettivo è mantenere la CPU attiva il più possibile, tipicamente tra il 40% e il 90% a seconda del carico di lavoro. Un basso utilizzo indica tempi morti significativi, mentre un utilizzo troppo elevato senza equilibrio può causare congestione del sistema.

2. **Throughput**: rappresenta il numero di processi completati per unità di tempo. Un elevato throughput indica che il sistema è in grado di completare molti processi rapidamente, aumentando l'efficienza complessiva.

3. **Turnaround Time**: è il tempo totale impiegato da un processo dal momento della sua sottomissione fino al completamento. Include tutti i tempi di attesa in coda, i burst di CPU e le operazioni di I/O. Minimizare il turnaround time migliora la soddisfazione dell'utente e l'efficienza globale.

4. **Tempo di attesa (*Waiting Time*)**: è il tempo totale trascorso da un processo nella *ready queue*, in attesa di essere eseguito dalla CPU. Minimizare il tempo di attesa è essenziale per ridurre ritardi percepiti e garantire equità tra i processi.

5. **Tempo di risposta (*Response Time*)**: indica il ritardo tra la sottomissione di un processo e la prima risposta fornita dal sistema. È particolarmente importante nei sistemi interattivi, dove l'utente percepisce direttamente la reattività del sistema. Un tempo di risposta prevedibile e breve è spesso più apprezzato della pura velocità di esecuzione.

7.3 Obiettivi della pianificazione

- **Massimizzare**: CPU utilization e throughput, assicurando che la CPU lavori costantemente ed esegua molti processi in breve tempo.

- **Minimizzare**: turnaround time, waiting time e response time, riducendo i ritardi per ogni processo e migliorando la reattività del sistema.

7.4 Idea chiave

Nei sistemi reali, la consistenza e la prevedibilità del tempo di risposta sono spesso più importanti della pura velocità di esecuzione. Un algoritmo di scheduling efficace deve bilanciare equità, efficienza e reattività, adattandosi alle caratteristiche dei processi (CPU-bound vs I/O-bound) e agli obiettivi del sistema.

7.5 Osservazioni

- Spesso esiste un compromesso tra throughput e tempi di risposta: massimizzare il throughput può aumentare il tempo di attesa dei processi interattivi.
- Le metriche vanno considerate insieme, non singolarmente, per ottenere una valutazione completa della performance dello scheduler.
- Algoritmi diversi privilegiano metriche diverse: ad esempio, Shortest Job First minimizza turnaround e waiting time, mentre Round Robin migliora il tempo di risposta nei sistemi interattivi.

8 Schedulazione First-Come, First-Served (FCFS)

8.1 Introduzione

Il **First-Come, First-Served (FCFS)** è il più semplice algoritmo di pianificazione della CPU. La logica è diretta: i processi vengono eseguiti nell'ordine in cui arrivano nella *ready queue*, secondo una strategia **FIFO (First-In-First-Out)**.

- La CPU viene assegnata al processo che ha richiesto per primo l'esecuzione.
- L'algoritmo è semplice da implementare e richiede una struttura dati minimale, come una lista concatenata ordinata per ordine di arrivo.
- Tuttavia, FCFS può comportare un **tempo medio di attesa elevato**, specialmente quando processi CPU-bound precedono processi I/O-bound brevi.

8.2 Esempio pratico

Consideriamo tre processi con i seguenti *burst time* (tempo di CPU richiesto):

Processo	Burst Time (ms)
P1	24
P2	3
P3	3

Seguendo FCFS, i processi vengono eseguiti nell'ordine P1, P2, P3.

8.3 Analisi del problema

- L'ordine di arrivo dei processi influisce direttamente sul tempo medio di attesa. Se un processo lungo (CPU-bound) arriva prima, tutti i processi successivi devono attendere, anche se hanno burst di CPU brevi.
- Questo fenomeno è noto come **convoy effect**: un processo CPU-bound "convoglia" i processi più brevi, specialmente quelli I/O-bound, causando ritardi ingiustificati.
- FCFS *non è ottimale* in termini di tempo medio di attesa e turnaround time, poiché non tiene conto della durata dei burst dei processi.

8.4 Osservazioni

- FCFS è semplice da capire e implementare, quindi può essere adatto per sistemi batch o con pochi processi.
- Nei sistemi interattivi o con alta variabilità dei burst, l'algoritmo può causare inefficienza e lunga attesa per i processi brevi.
- Miglioramenti possono essere ottenuti utilizzando algoritmi che considerano la lunghezza del burst, come lo *Shortest Job First (SJF)*, riducendo il tempo medio di attesa.

9 Schedulazione Shortest Job First (SJF)

9.1 Introduzione

Il **Shortest Job First (SJF)** è un algoritmo di pianificazione della CPU che assegna la CPU al processo che ha il *burst di CPU più breve* previsto tra quelli pronti. Questo approccio è anche noto come **Shortest Next CPU Burst** e si basa sull'idea di minimizzare il tempo medio di attesa dei processi.

- La CPU viene assegnata al processo con il burst di CPU più breve.
- Se due processi hanno burst uguali, si applica la politica *First-Come, First-Served (FCFS)* per stabilire quale processo eseguire per primo.
- SJF può essere implementato in due varianti:
  - **Non-preemptive SJF**: una volta assegnata la CPU, il processo viene eseguito fino al termine del burst.
  - **Preemptive SJF (Shortest Remaining Time First, SRTF)**: se un nuovo processo arriva con un burst più breve del processo in esecuzione, la CPU viene preemptata e assegnata al nuovo processo.

9.2 Esempio pratico

Consideriamo quattro processi con i seguenti burst di CPU (in millisecondi):

Processo	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

Procedura SJF non-preemptive:

1. Si seleziona il processo con il burst più breve: P4 (3 ms) → eseguito per primo.
2. Tra i processi rimanenti (P1, P2, P3), il burst più breve è P1 (6 ms) → eseguito successivamente.
3. Poi P3 (7 ms), infine P2 (8 ms).

9.3 Idea chiave

Il SJF è **ottimale** in termini di tempo medio di attesa: minimizza la media del tempo di turnaround e del tempo di attesa rispetto ad altri algoritmi non-preemptive. Assegnare la CPU ai processi più brevi riduce il tempo complessivo di attesa dei processi nella coda dei pronti.

9.4 Osservazioni

- La difficoltà principale di SJF è prevedere la lunghezza del prossimo burst di CPU. Nei sistemi reali, si utilizzano stime basate su burst precedenti o algoritmi predittivi.
- Il SJF preemptive (Shortest Remaining Time First) migliora ulteriormente il tempo medio di attesa quando arrivano nuovi processi con burst più brevi.
- Sebbene ottimale per la media dei tempi, SJF può portare a **starvation** per processi con burst lunghi se arrivano costantemente processi brevi.
- L'algoritmo è particolarmente efficace in sistemi batch o con processi di lunghezza variabile nota, mentre nei sistemi interattivi richiede predizioni accurate dei burst.

10 Approssimazione dello Shortest Job First (SJF)

10.1 Introduzione

Il **Shortest Job First (SJF)** è ottimale per minimizzare il tempo medio di attesa, ma presenta un problema fondamentale: non è possibile conoscere in anticipo la lunghezza dei burst di CPU futuri. Per ovviare a questo problema, i sistemi operativi utilizzano tecniche di predizione, che stimano la durata del prossimo burst basandosi sulle informazioni passate. Questo approccio è noto come **approssimazione dello SJF**.

10.2 Predizione dei burst futuri tramite media esponenziale

La tecnica più comune per stimare il prossimo burst di CPU utilizza la **media esponenziale**, definita dalla seguente formula:

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$$

dove:

- τ_{n+1}: previsione della durata del prossimo burst di CPU.
- t_n: durata effettiva dell'ultimo burst di CPU.
- τ_n: previsione precedente.
- α: parametro di ponderazione, con $0 \leq \alpha \leq 1$, che bilancia il peso tra informazioni recenti e passate.

10.3 Significato del parametro α

- $\alpha = 0$: la previsione si basa completamente sulla storia passata dei burst; il valore precedente τ_n ha pieno peso. Questo riduce la sensibilità ai cambiamenti recenti ma fornisce una stima più stabile.
- $\alpha = 1$: la previsione si basa esclusivamente sull'ultimo burst misurato t_n. Questo rende la stima molto reattiva ai cambiamenti recenti, ma meno stabile.
- $\alpha = 0.5$: la previsione attribuisce uguale peso all'ultimo burst e alla storia passata, bilanciando reattività e stabilità.

10.4 Idea chiave

La predizione tramite media esponenziale permette di approssimare lo SJF anche quando la durata dei burst futuri è sconosciuta. Pur non essendo perfetta, questa tecnica consente di ottenere prestazioni molto vicine all'ottimo, minimizzando il tempo medio di attesa senza conoscere i burst futuri con certezza.

10.5 Osservazioni

- La scelta di α dipende dalla variabilità dei processi: sistemi con burst stabili possono usare valori bassi di α, mentre sistemi con burst molto variabili beneficiano di valori più alti.
- La media esponenziale può essere calcolata in modo incrementale, evitando di memorizzare l'intera storia dei burst.
- Questo approccio è alla base di scheduler predittivi reali nei sistemi operativi moderni.

11 SJF Preemptive vs Non-Preemptive

11.1 Introduzione

Il **Shortest Job First (SJF)** può essere implementato in due modalità principali: **non-preemptive** e **preemptive**. La differenza fondamentale riguarda la possibilità di interrompere un processo in esecuzione quando arriva un nuovo processo con burst più breve.

11.2 SJF Non-Preemptive

- Una volta che un processo ottiene la CPU, completa il suo burst senza essere interrotto.
- Lo scheduler seleziona sempre il processo con il burst più breve tra quelli pronti, ma non può preemptare il processo in esecuzione.
- Questa modalità è più semplice da implementare e riduce il numero di context switch, ma può aumentare il tempo medio di attesa se arrivano nuovi processi con burst più brevi mentre un processo lungo è in esecuzione.

11.3 SJF Preemptive (Shortest Remaining Time First, SRTF)

- La CPU può essere assegnata a un nuovo processo se questo ha un burst residuo più breve rispetto al processo corrente.
- Ogni volta che arriva un nuovo processo, lo scheduler confronta il burst residuo del processo in esecuzione con il burst previsto del nuovo processo. Se il nuovo processo è più breve, viene preemptato il processo corrente e la CPU passa al nuovo processo.
- Questa modalità migliora la reattività del sistema e riduce il tempo medio di attesa.

11.4 Esempio di confronto

Considerando lo stesso insieme di processi usato negli esempi precedenti, i tempi medi di attesa risultano:

- **Preemptive SJF (SRTF)**: tempo medio di attesa = 6.5 ms
- **Non-Preemptive SJF**: tempo medio di attesa = 7.75 ms

11.5 Idea chiave

La versione preemptive dello SJF (*SRTF*) ottiene generalmente migliori prestazioni rispetto alla non-preemptive, riducendo il tempo medio di attesa e migliorando il throughput, soprattutto in presenza di processi con burst di lunghezza variabile.

11.6 Osservazioni

- La preemption introduce un leggero overhead dovuto ai context switch, ma in molti casi il miglioramento del tempo medio di attesa compensa ampiamente questo costo.
- In sistemi interattivi o con alta variabilità dei burst, lo SJF preemptive è preferibile per mantenere la CPU sempre occupata dai processi più brevi.
- Lo SJF non-preemptive può ancora essere utile in sistemi batch o con processi prevedibili, dove la semplicità dell'algoritmo è un vantaggio.

12 Schedulazione Round Robin (RR)

12.1 Introduzione

Il **Round Robin (RR)** è un algoritmo di pianificazione della CPU progettato per sistemi interattivi. Esso combina la semplicità del *First-Come, First-Served (FCFS)* con il meccanismo di preemption, garantendo che ogni processo riceva una quantità fissa di tempo per l'esecuzione, detta **time quantum** o *time slice*.

- Ogni processo in esecuzione riceve un quantum di tempo definito.
- La *ready queue* è organizzata come una coda circolare (FIFO), in cui i processi completati o preemptati vengono reinseriti alla fine della coda.
- Se il burst di CPU di un processo è maggiore del quantum, il processo viene interrotto alla scadenza del quantum e spostato in fondo alla coda.
- Se il burst è minore del quantum, il processo completa l'esecuzione prima che il quantum scada.

12.2 Dipendenza dalle dimensioni del quantum

Le prestazioni del Round Robin dipendono fortemente dalla dimensione del quantum:

- **Quantum molto grande**: il comportamento tende a quello di FCFS, con bassa preemption e pochi context switch.
- **Quantum molto piccolo**: aumenta il numero di context switch, con conseguente overhead elevato e riduzione dell'efficienza della CPU.

12.3 Esempi numerici

12.3.1 Un processo di 10 unità

- **Quantum = 12**: il processo completa l'esecuzione in un solo quantum $\rightarrow$ 0 context switch.
- **Quantum = 6**: il processo necessita di due quantum $\rightarrow$ 1 context switch.
- **Quantum = 1**: il processo necessita di dieci quantum $\rightarrow$ 9 context switch.

12.3.2 Tre processi di 10 unità ciascuno

- **Quantum = 1**: tempo medio di turnaround = 29 unità.
- **Quantum = 10**: tempo medio di turnaround = 20 unità.

12.4 Idea chiave

Il Round Robin garantisce equità, poiché ogni processo riceve periodicamente la CPU, ma le prestazioni dipendono dalla scelta del quantum. Un quantum troppo piccolo aumenta l'overhead dei context switch e può peggiorare il tempo medio di turnaround, mentre un quantum troppo grande riduce la reattività del sistema e tende a comportarsi come FCFS.

12.5 Osservazioni

- Il tempo di turnaround non migliora necessariamente aumentando il quantum: l'ottimizzazione si ottiene quando la maggior parte dei processi completa il burst entro un solo quantum.
- Un numero elevato di context switch comporta maggiore overhead, aumentando il tempo medio di turnaround.
- La scelta del quantum rappresenta un compromesso tra reattività del sistema (quantum breve) ed efficienza della CPU (quantum lungo).

13 Schedulazione a Priorità (Priority Scheduling)

13.1 Introduzione

La **schedulazione a priorità** è un'estensione generale dello SJF, in cui ogni processo viene associato a un **valore di priorità**. La CPU viene assegnata al processo con la priorità più alta, secondo criteri stabiliti dallo scheduler.

- Lo **SJF** può essere considerato un caso particolare di schedulazione a priorità: in SJF, la priorità di un processo è inversamente proporzionale alla lunghezza prevista del burst di CPU.
- In generale, nella schedulazione a priorità:
  - Processi con priorità più alta vengono eseguiti per primi.
  - Se due processi hanno la stessa priorità, si applica la politica *FCFS* per determinare l'ordine di esecuzione.

13.2 Esempio di priorità basata sul burst

Nel caso dello SJF predittivo, la priorità di un processo può essere definita come:

$$\text{priorità} = \frac{1}{\tau_{\text{prevista}}}$$

dove τ_{prevista} è la durata stimata del prossimo burst di CPU.

- Burst più lunghi $\rightarrow$ priorità più bassa (valore numerico maggiore).
- Burst più brevi $\rightarrow$ priorità più alta (valore numerico minore in questo esempio).

13.3 Problema della starvation (invecchiamento)

Un problema significativo della schedulazione a priorità è la **starvation**, ovvero la possibilità che processi a bassa priorità restino indefinitamente in attesa:

- Se continuano ad arrivare processi ad alta priorità, la CPU non viene mai assegnata ai processi a bassa priorità, anche se sono tecnicamente pronti.
- Questo fenomeno è particolarmente critico nei sistemi con carico elevato, dove i processi a priorità bassa possono non eseguire mai.
- La starvation compromette l'equità del sistema e può causare ritardi significativi per determinati task.

13.4 Idea chiave

La schedulazione a priorità permette di ottimizzare l'uso della CPU e ridurre il tempo di attesa dei processi più importanti, ma introduce il rischio che processi meno importanti non vengano mai eseguiti. È fondamentale bilanciare priorità e fairness.

13.5 Osservazioni

- Una tecnica comune per mitigare la starvation è l'**aging**, in cui la priorità di un processo aumenta progressivamente con il tempo di attesa, garantendo che anche i processi a priorità bassa vengano eventualmente eseguiti.
- Nei sistemi reali, le priorità possono essere statiche (fisse) o dinamiche (modificate durante l'esecuzione), a seconda delle caratteristiche dei processi e degli obiettivi del sistema.
- La scelta di una politica di priority scheduling deve considerare sia l'efficienza della CPU che l'equità tra processi di diversa importanza.

14 Risoluzione della Starvation: Aging e Priority + Round Robin

14.1 Introduzione

La **starvation** è un problema comune nei sistemi con *priority scheduling*, dove processi a bassa priorità possono non essere mai eseguiti. Una soluzione efficace combina due tecniche principali:

1. **Aging**: aumento progressivo della priorità dei processi che attendono da molto tempo.
2. **Hybrid Priority + Round Robin**: utilizzo della schedulazione a priorità per determinare quale processo eseguire e Round Robin per gestire processi con priorità uguale.

14.2 Esempio pratico

Consideriamo cinque processi con burst e priorità iniziali come segue:

Processo	Burst (ms)	Priorità iniziale
P	4	3
P	5	2
P	8	2
P	7	1
P	3	3

14.3 Aging

- L'**aging** consiste nell'aumentare progressivamente la priorità di un processo in attesa.
- Ad esempio, se la priorità va da 127 (bassa) a 0 (alta), un incremento di +1 per secondo permette a un processo inizialmente a priorità 127 di raggiungere la priorità massima 0 in circa 2 minuti.
- Questo garantisce che nessun processo resti indefinitamente in attesa, risolvendo la starvation.

14.4 Schedulazione ibrida: Priority + Round Robin

- **Priorità**: il processo con priorità più alta viene eseguito per primo.
- **Round Robin**: quando più processi hanno la stessa priorità, vengono schedulati con RR utilizzando un *time quantum* definito (es. 2 ms).
- Questo approccio combina equità tra processi di pari priorità e rispetto delle priorità globali del sistema.

14.5 Idea chiave

L'invecchiamento (*aging*) previene la starvation aumentando gradualmente la priorità dei processi in attesa. La combinazione di **Priority Scheduling** con **Round Robin** per processi a pari priorità garantisce sia la correttezza della priorità sia l'equità tra processi simili, ottimizzando sia il tempo di attesa che il throughput.

14.6 Osservazioni

- La velocità di aging deve essere scelta con attenzione: troppo lenta non risolve tempestivamente la starvation, troppo rapida può invertire l'ordine di priorità naturale.
- La schedulazione ibrida è particolarmente utile in sistemi interattivi con processi misti (CPU-bound e I/O-bound), dove sia l'equità che la reattività sono importanti.
- Questo approccio consente al sistema operativo di mantenere alte prestazioni complessive, prevenendo blocchi prolungati di processi a bassa priorità.

15 Schedulazione a Code Multilivello (Multilevel Queue Scheduling)

15.1 Introduzione

La **schedulazione a code multilivello** è una tecnica di CPU scheduling progettata per sistemi in cui i processi possono essere raggruppati in classi diverse, tipicamente basate sulla priorità o sul tipo di processo (CPU-bound, I/O-bound, interattivo, batch, ecc.). L'obiettivo è ottimizzare l'efficienza dello scheduler e ridurre il tempo di ricerca nella coda dei pronti.

15.2 Struttura del sistema

- Ogni livello di priorità ha la propria **ready queue** separata.
- Lo scheduler seleziona sempre un processo dalla coda di priorità più alta che contenga processi pronti.
- Se più processi condividono la stessa priorità, essi vengono schedulati con una politica *Round Robin* all'interno della coda.

15.3 Vantaggi principali

- **Efficienza**: evitare di scansionare una singola coda lunga riduce la complessità dello scheduler, eliminando la necessità di un'operazione $O(n)$ per trovare il processo successivo.
- **Gestione per classi**: permette di applicare politiche differenti a diverse classi di processi, ad esempio Round Robin per processi interattivi e FCFS per processi batch.

15.4 Priorità statica

- In questa implementazione, la priorità di un processo è **statica**: ogni processo rimane nella coda a cui è stato assegnato per tutta la durata della sua vita.
- Questo garantisce prevedibilità nell'esecuzione dei processi, ma limita la flessibilità se un processo cambia comportamento (ad esempio, da CPU-bound a I/O-bound).

15.5 Idea chiave

Le **Multilevel Queue** permettono di organizzare i processi in base alla priorità e al tipo, evitando la scansione di una singola coda e consentendo allo scheduler di concentrarsi rapidamente sui processi più importanti, garantendo efficienza e gestione differenziata.

15.6 Classificazione dei processi e politiche per coda

- I processi possono essere raggruppati per **tipo**, ad esempio:
  - **Foreground (interattivi)**: processi che richiedono risposta rapida all'utente.
  - **Background (batch)**: processi che eseguono elaborazioni in background, con minore sensibilità ai tempi di risposta.
- Ogni coda può utilizzare un **algoritmo di scheduling differente**:
  - La coda foreground spesso utilizza *Round Robin (RR)* per garantire equità e bassa latenza.
  - La coda background tipicamente utilizza *First-Come, First-Served (FCFS)*, privilegiando la semplicità e l'esecuzione completa dei processi.
- La **preemption a priorità fissa** tra code garantisce che i processi nella coda con priorità più alta possano interrompere quelli nelle code di priorità più bassa.
- In alcune implementazioni avanzate, è possibile assegnare **quote di CPU (CPU time shares)** a ciascuna coda, bilanciando l'uso della CPU tra code diverse e assicurando che nessuna classe di processi monopolizzi la CPU.

15.7 Idea chiave

La schedulazione a code multilivello permette di differenziare il trattamento dei processi in base al tipo e alla priorità, applicando algoritmi diversi per massimizzare sia la reattività dei processi interattivi sia l'efficienza dei processi batch. La preemption tra code e l'assegnazione di quote di CPU assicurano equità e controllo sul tempo di esecuzione complessivo.

15.8 Osservazioni

- La combinazione di diverse politiche per coda consente di ottimizzare le prestazioni complessive del sistema, bilanciando reattività, throughput e equità.
- L'assegnazione di quote di CPU per coda è utile in sistemi multitasking complessi, dove processi di diverso tipo devono coesistere senza starvazione reciproca.
- La preemption tra code riduce il rischio che processi interattivi vengano bloccati da processi batch a priorità inferiore.

16 Schedulazione a Code Multilivello con Feedback (Multilevel Feedback Queue, MLFQ)

16.1 Introduzione

La **schedulazione a code multilivello con feedback (MLFQ)** rappresenta l'approccio più flessibile e complesso per il CPU scheduling. A differenza della semplice *Multilevel Queue Scheduling*, i processi non rimangono fissi in una coda: possono **spostarsi tra code di priorità diverse** in base al loro comportamento e alle regole dello scheduler.

16.2 Meccanismo di feedback

- I processi **CPU-bound**, che richiedono lunghe esecuzioni, vengono spostati verso code a priorità inferiore, riducendo la loro possibilità di monopolizzare la CPU.
- I processi **I/O-bound** e interattivi, che alternano brevi burst di CPU e attese per I/O, rimangono in code ad alta priorità, migliorando la reattività del sistema.
- L'**aging** viene applicato per prevenire la starvation dei processi a bassa priorità, promuovendoli gradualmente a code superiori se restano troppo a lungo in attesa.

16.3 Definizione della MLFQ

Un sistema MLFQ è definito da diversi elementi chiave:

1. **Numero di code**: determina quanti livelli di priorità esistono.
2. **Algoritmo di scheduling per coda**: ogni coda può usare FCFS, RR o altre politiche.
3. **Regole di promozione e retrocessione**: definiscono quando un processo passa a una coda di priorità più alta o più bassa in base al comportamento osservato.
4. **Regola di assegnazione iniziale**: stabilisce in quale coda un processo viene inserito al momento della creazione.

16.4 Idea chiave

La MLFQ adatta dinamicamente le priorità dei processi in base al loro utilizzo della CPU e al comportamento interattivo, combinando i vantaggi di priorità elevate per processi reattivi e limitando l'uso eccessivo della CPU da parte di processi CPU-bound. Questo rende la MLFQ la strategia più flessibile e potente per sistemi multitasking complessi.

16.5 Osservazioni

- La MLFQ richiede una progettazione accurata: un numero appropriato di code, politiche per coda e regole di feedback sono fondamentali per ottenere buone prestazioni.
- L'aging è essenziale per garantire equità e prevenire starvation, specialmente in sistemi con processi di priorità variabile.
- La complessità della MLFQ è maggiore rispetto alle code multilivello statiche, ma consente di bilanciare reattività, throughput e utilizzo della CPU in modo ottimale.
- I sistemi operativi moderni spesso implementano varianti semplificate di MLFQ per gestire carichi misti di processi interattivi e batch.

17 Schedulazione dei Thread: User-Level vs Kernel-Level

17.1 Introduzione

Nei sistemi multitasking moderni, la CPU può essere condivisa non solo tra processi, ma anche tra **thread**. È importante distinguere tra **thread a livello utente** e **thread a livello kernel**, poiché la gestione e la schedulazione avvengono in modi differenti:

- Il sistema operativo gestisce e schedula direttamente **solo i thread a livello kernel**, non i processi nel loro insieme.
- I **thread a livello utente** sono gestiti dalla libreria di thread nel processo utente. Lo scheduler a livello utente decide quale thread del processo eseguire, ma per ottenere tempo CPU devono essere mappati a thread kernel.

17.2 Mappatura dei thread

- Ogni thread a livello utente deve essere associato a un thread kernel per poter essere eseguito sul processore.
- I modelli di mappatura includono:
  - **Many-to-One**: molti thread utente condividono un solo thread kernel.
  - **Many-to-Many**: molti thread utente possono essere mappati su molti thread kernel.
  - **One-to-One**: ogni thread utente corrisponde a un thread kernel.

17.3 Scope di schedulazione

Esistono due principali **scope di competizione** per la schedulazione dei thread:

1. **Process-Contention Scope (PCS)**:
   - La competizione avviene *tra i thread dello stesso processo*.
   - Tipicamente utilizzato nei modelli many-to-one e many-to-many.
   - Lo scheduler a livello utente decide quale thread del processo eseguire, senza interferire con altri processi.
2. **System-Contention Scope (SCS)**:
   - La competizione avviene *tra tutti i thread del sistema*, indipendentemente dal processo di appartenenza.
   - Lo scheduler a livello kernel seleziona quale thread kernel riceve la CPU in un dato istante.
   - Garantisce equità a livello di sistema e bilanciamento del carico tra processi multipli.

17.4 Idea chiave

I **thread a livello utente** competono solo all'interno del loro processo, gestiti dalla libreria di thread, mentre i **thread a livello kernel** competono su scala di sistema. La distinzione è fondamentale per comprendere le prestazioni e la concorrenza nei sistemi multi-threaded.

17.5 Osservazioni

- Nei modelli many-to-one, un thread utente in esecuzione blocca tutti gli altri thread del processo se il thread kernel associato è occupato o in attesa di I/O.
- Nei modelli many-to-many, la flessibilità aumenta, poiché più thread utente possono essere eseguiti in parallelo su più thread kernel.
- Lo scheduler del kernel utilizza lo scope SCS per bilanciare il carico tra processi e thread a livello di sistema, mentre lo scheduler a livello utente utilizza PCS per gestire l'esecuzione interna ai processi.
- La corretta combinazione di PCS e SCS ottimizza l'utilizzo della CPU e riduce i tempi di attesa dei thread interattivi.

18 Pthreads: Contention Scope

18.1 Introduzione

La libreria **Pthreads** (POSIX Threads) consente di creare e gestire thread in sistemi UNIX-like. Un aspetto fondamentale è la possibilità di scegliere lo **scope di competizione** di ciascun thread al momento della creazione:

- **Process-Contention Scope (PCS)**: i thread competono solo all'interno del loro processo.
- **System-Contention Scope (SCS)**: i thread competono su scala di sistema, tra tutti i thread kernel.

18.2 Policy disponibili

- **PTHREAD_SCOPE_PROCESS**: utilizza PCS. I thread competono solo tra loro nel processo corrente, tipico dei modelli many-to-one e many-to-many.
- **PTHREAD_SCOPE_SYSTEM**: utilizza SCS. I thread competono con tutti gli altri thread del sistema, tipico dei modelli one-to-one o many-to-many configurati in modo appropriato.

18.3 Comportamento nei sistemi Many-to-Many

- **PCS**: i thread utente sono mappati su LWPs (Lightweight Processes) disponibili; la competizione è limitata all'interno del processo.
- **SCS**: ogni thread utente può avere un LWP dedicato, comportandosi in pratica come un modello many-to-one rispetto al kernel.

18.4 Funzioni API per gestire lo scope

- Impostare lo scope di un thread prima della creazione:

```
pthread_attr_setscope(attr, scope)
```

- Leggere lo scope attuale di un thread:

```
pthread_attr_getscope(attr, scope)
```

18.5 Idea chiave

Pthreads permette di selezionare lo scope di competizione più adatto alle esigenze dell'applicazione. Lo scope PCS limita la competizione all'interno del processo, utile per thread cooperativi, mentre lo scope SCS consente ai thread di competere a livello di sistema, aumentando la concorrenza e la parallelizzazione.

18.6 Osservazioni

- La scelta dello scope influisce sulle prestazioni e sul grado di parallelismo dei thread.
- In modelli many-to-many, PCS può ridurre l'overhead del kernel ma limita la concorrenza globale, mentre SCS aumenta il parallelismo ma può incrementare i context switch.
- L'uso corretto delle API di Pthreads garantisce controllo preciso sul comportamento dei thread e sulla gestione delle risorse CPU.

19 Schedulazione su Multiprocessore

19.1 Introduzione

Finora abbiamo studiato principalmente algoritmi di schedulazione per **CPU single-core**. L'introduzione di **più CPU** o core consente di condividere il carico di lavoro tra più processori, aumentando le prestazioni complessive, ma comporta anche una maggiore complessità nella gestione dei processi.

19.2 Significato moderno di "multiprocessore"

Nei sistemi moderni, il termine **multiprocessore** può indicare diverse architetture:

- **CPU multicore**: più core fisici su un singolo chip.
- **Core multithreaded**: core che supportano più thread in parallelo.
- **Sistemi NUMA (Non-Uniform Memory Access)**: memoria condivisa con latenza variabile tra i processori.
- **Multiprocessori eterogenei**: CPU con prestazioni o capacità diverse.

19.3 Multiprocessing asimmetrico (Asymmetric Multiprocessing)

- Un solo processore, detto **master**, gestisce la schedulazione, le operazioni di I/O e le funzioni del sistema operativo.
- Gli altri processori eseguono solo codice utente.
- Vantaggi: semplifica la condivisione dei dati e la gestione delle risorse.
- Svantaggi: il master CPU può diventare un **collo di bottiglia** se sovraccarico.

19.4 Multiprocessing simmetrico (Symmetric Multiprocessing, SMP)

- Ogni processore può auto-schedulare i propri processi.
- Due approcci principali per le **ready queue**:
  1. **Coda condivisa**: tutti i processori attingono dalla stessa coda dei pronti. Necessita di lock per sincronizzazione, con possibile contesa.
  2. **Code private per processore**: ciascun processore ha la propria coda dei pronti, evitando contesa e migliorando le prestazioni. Questo approccio è comune nella pratica.

19.5 Multicore su singolo chip

- Ogni core è visto dal sistema operativo come una CPU separata.
- Vantaggi principali:
  - Miglior performance complessiva.
  - Minore consumo energetico rispetto a più CPU single-chip.

19.6 Sfide nello scheduling su multiprocessore

- **Ritardi di accesso alla memoria**: la CPU può restare inattiva durante attese di dati, fino al 50% del tempo in alcuni casi.
- **Cache miss e latenza di memoria**: rendono la predizione dei tempi di esecuzione più complessa.
- Lo scheduler deve considerare non solo la disponibilità della CPU, ma anche l'accesso ai dati e la località della memoria per ottimizzare le prestazioni.

19.7 Idea chiave

La schedulazione su sistemi multiprocessore richiede strategie avanzate per bilanciare il carico tra CPU, ridurre la contesa sulle code dei pronti e minimizzare i ritardi dovuti alla memoria. La scelta tra SMP, code condivise o private, e l'uso di multicore fisici sono elementi chiave per ottimizzare le prestazioni e l'efficienza energetica del sistema.

19.8 Osservazioni

- La gestione della memoria e della cache diventa critica in sistemi NUMA o multicore, poiché un accesso non locale può causare ritardi significativi.
- L'uso di code private per processore riduce l'overhead da lock e contesa, ma richiede tecniche di **load balancing** per evitare che alcune CPU rimangano inattive mentre altre sono sovraccariche.
- Le architetture eterogenee complicano ulteriormente le decisioni di scheduling, poiché alcuni core possono essere più adatti di altri per processi specifici.

20 Chip Multithreading (CMT)

20.1 Introduzione

Il **Chip Multithreading (CMT)** è una tecnica che permette a un singolo core di CPU di eseguire più thread hardware in parallelo. L'idea fondamentale è sfruttare i cicli di inattività dovuti a stall della memoria, migliorando l'utilizzo del core.

- Più **hardware thread** condividono un singolo core fisico.
- Se un thread si blocca per accesso a memoria o altre operazioni a lunga latenza, il core può passare all'esecuzione di un altro thread pronto.
- Ogni hardware thread possiede i propri registri e il proprio *instruction pointer*.
- Il sistema operativo percepisce ogni hardware thread come una **CPU logica** separata.

20.2 Tipi di Multithreading

20.2.1 Coarse-Grained Multithreading (Multithreading Grossolano)

- Lo switch tra thread avviene solo in corrispondenza di eventi a lunga latenza, come uno **stall di memoria**.
- Lo svantaggio principale è l'elevato **costo di switch**, poiché la pipeline del core deve essere svuotata (*flush*) e ricaricata con il nuovo thread.
- Il nuovo thread inizia quindi a riempire la pipeline, causando un breve ritardo prima che possa eseguire pienamente.

20.2.2 Fine-Grained (Interleaved) Multithreading (Multithreading Fine/Interlacciato)

- Lo switch tra thread avviene frequentemente, a granulosità fine, ad ogni ciclo o pochi cicli di clock.
- Richiede hardware dedicato per eseguire switch rapidi e gestire più *instruction pointers* contemporaneamente.
- Il costo di switch è basso, permettendo al core di mantenere un utilizzo elevato e ridurre i cicli vuoti.

20.3 Idea chiave

Il CMT aumenta l'**efficienza del core** riducendo i periodi di inattività dovuti a stall di memoria. La scelta tra multithreading grossolano o fine dipende dal tipo di workload e dalla capacità hardware di gestire switch rapidi tra thread.

20.4 Osservazioni

- Nei sistemi con molti thread a bassa latenza, il multithreading fine è preferibile per ridurre il tempo di inattività e massimizzare throughput.
- Nei sistemi con pochi stall di lunga durata, il multithreading grossolano può essere sufficiente e più semplice da implementare.
- Il CMT non aumenta la frequenza di esecuzione di un singolo thread, ma migliora l'utilizzo complessivo del core condiviso tra più thread.
- Dal punto di vista del sistema operativo, ogni thread hardware è trattato come una CPU logica, semplificando la gestione del scheduling multi-thread a livello di OS.

21 Schedulazione a Due Livelli nei Sistemi Multicore Multithreaded

21.1 Introduzione

Nei sistemi multicore con **chip multithreading**, le risorse del core sono condivise tra più hardware thread, ma ogni core può eseguire **solo un hardware thread alla volta**. Per gestire questa complessità, viene utilizzata una **schedulazione a due livelli**.

21.2 I due livelli di schedulazione

1. **Livello OS (Software Thread Scheduling)**:
   - Il sistema operativo seleziona quale **software thread** (thread gestito a livello utente o kernel) deve essere assegnato a ciascun hardware thread disponibile.
   - Questo livello decide l'assegnazione globale dei thread su tutti i core e tutte le CPU logiche.
2. **Livello Core (Hardware Thread Scheduling)**:
   - Ogni core decide quale dei propri hardware thread pronti deve essere eseguito in un dato ciclo.
   - La scelta può essere basata su round-robin, priorità dinamica o altre politiche integrate nell'hardware.

21.3 Esempi di implementazione

- **UltraSPARC T3**: utilizza uno *round-robin hardware thread scheduling*, in cui il core passa ciclicamente tra i thread hardware pronti.
- **Intel Itanium**: impiega una *dynamic urgency* per ogni thread (valori da 0 a 7), cambiando thread su eventi specifici e eseguendo sempre quello con urgenza più alta.

21.4 Idea chiave

La schedulazione a due livelli permette di ottimizzare l'utilizzo delle risorse dei core multi-threaded, separando la responsabilità del sistema operativo (assegnazione dei software thread) dalla gestione hardware dei thread fisici. Questo approccio migliora sia il throughput globale sia la reattività dei thread interattivi.

21.5 Osservazioni

- Il livello hardware può reagire rapidamente agli stall della pipeline o della memoria senza coinvolgere il sistema operativo, riducendo i tempi morti.
- Il livello software OS mantiene il controllo complessivo sul bilanciamento del carico tra core e processori.
- La combinazione di questi due livelli permette di ottenere alta efficienza sia per thread CPU-bound che per thread I/O-bound.
- Le politiche hardware di scheduling influenzano direttamente il throughput e la latenza dei thread, mentre le politiche software determinano l'equità e la distribuzione complessiva del lavoro tra i core.

22 Bilanciamento del Carico nei Sistemi SMP (Load Balancing in SMP Systems)

22.1 Introduzione

Nei sistemi **Symmetric Multiprocessing (SMP)** con **code private per processore**, il bilanciamento del carico è fondamentale per garantire un utilizzo uniforme della CPU e massimizzare il throughput complessivo.

- L'obiettivo è **distribuire equamente il lavoro tra tutti i processori**.
- Nei sistemi con **coda condivisa**, il bilanciamento è intrinseco: ogni processore prende processi dalla stessa coda, quindi non è necessario un meccanismo aggiuntivo.
- Nei sistemi con **code private**, invece, alcuni processori possono diventare sovraccarichi mentre altri restano inattivi, rendendo indispensabile il bilanciamento del carico.

22.2 Strategie di migrazione dei processi

Due approcci principali vengono utilizzati per bilanciare il carico tra i processori:

1. **Push Migration (Migrazione a spinta)**:
   - Un processore **sovraccarico** spinge i processi meno prioritari verso processori meno occupati.
   - Garantisce che nessun processore rimanga troppo carico rispetto agli altri.
2. **Pull Migration (Migrazione a trazione)**:
   - Un processore **inattivo o sotto-utilizzato** prende (*pull*) un processo dalla coda di un processore occupato.
   - Migliora l'efficienza dei processori inattivi e riduce i tempi di attesa complessivi.

22.3 Idea chiave

Il **load balancing** in sistemi SMP con code private garantisce che nessun processore sia sovraccarico o inattivo, migliorando le prestazioni globali. Le strategie *push* e *pull* offrono meccanismi complementari per distribuire dinamicamente i processi tra CPU.

22.4 Osservazioni

- La scelta tra push e pull dipende dall'architettura del sistema e dalle politiche dello scheduler.
- Push migration richiede che i processori sovraccarichi monitorino le code degli altri processori, mentre pull migration richiede che i processori inattivi controllino le code altrui.
- In sistemi reali, spesso si utilizzano combinazioni di entrambe le strategie per ottimizzare il bilanciamento del carico in tempo reale.
- Un bilanciamento efficiente riduce i tempi di attesa dei processi e aumenta il throughput, senza introdurre overhead eccessivo dovuto alle migrazioni.

23 Processor Affinity nei Sistemi SMP

23.1 Introduzione

La **Processor Affinity** è una tecnica utilizzata nei sistemi **Symmetric Multiprocessing (SMP)** per migliorare le prestazioni dei processi assegnati a specifici core. L'idea principale è mantenere un processo sulla stessa CPU durante la sua esecuzione, sfruttando la **località dei dati** nella cache.

23.2 Vantaggi dell'affinità

- Mantenere un processo sullo stesso processore consente di preservare i dati nella **cache calda** (*warm cache*), riducendo i cache miss e migliorando le prestazioni.
- Spostare un processo su un altro processore forza l'invalidazione della cache e il ricaricamento dei dati, causando overhead e riducendo l'efficienza complessiva.

23.3 Tipi di Processor Affinity

1. **Soft Affinity (affinità morbida)**:
   - Il sistema operativo cerca di mantenere il processo sulla stessa CPU, ma non è garantito.
   - Lo scheduler può comunque spostare il processo se necessario per bilanciare il carico tra i processori.
2. **Hard Affinity (affinità rigida)**:
   - Il processo specifica i processori sui quali può essere eseguito.
   - La migrazione è strettamente limitata alle CPU consentite, assicurando che la cache del processore sia sfruttata al massimo.

23.4 Idea chiave

La processor affinity migliora le prestazioni riducendo i cache miss e massimizzando la località dei dati. L'uso di affinità soft o hard dipende dalle esigenze di bilanciamento del carico e dall'importanza di preservare la cache per processi critici.

23.5 Osservazioni

- L'affinità soft è più flessibile e permette allo scheduler di bilanciare meglio il carico tra CPU, ma può occasionalmente degradare la località della cache.
- L'affinità hard massimizza l'efficienza della cache, ma può causare squilibri se alcune CPU diventano sovraccariche mentre altre sono inattive.
- Nei sistemi multicore moderni, combinare processor affinity con tecniche di **load balancing** e **multithreading hardware** può ottimizzare sia l'utilizzo della CPU che le prestazioni complessive.
- Alcuni sistemi operativi permettono di impostare l'affinità tramite API, ad esempio `sched_setaffinity()` in Linux.

24 Schedulazione della CPU in Tempo Reale (Real-Time CPU Scheduling)

24.1 Introduzione

La **schedulazione in tempo reale** riguarda sistemi in cui alcuni processi hanno requisiti temporali stringenti. L'obiettivo è garantire che task critici ricevano attenzione tempestiva, con priorità alle scadenze temporali.

24.2 Tipologie di sistemi real-time

1. **Soft Real-Time (tempo reale morbido)**:
   - I task critici ricevono **priorità più alta**, aumentando la probabilità di completamento entro tempi accettabili.
   - Non esiste una garanzia assoluta sulle scadenze; occasionali ritardi sono tollerabili.
   - Tipico in sistemi multimediali, interattivi o con QoS (Quality of Service) desiderato.
2. **Hard Real-Time (tempo reale rigido)**:
   - I task devono **completare entro scadenze rigorose**. Qualsiasi ritardo rende inutile l'esecuzione del task.
   - Richiede algoritmi di schedulazione deterministici, come *Rate-Monotonic Scheduling (RMS)* o *Earliest Deadline First (EDF)*.
   - Tipico in sistemi di controllo industriale, avionica, robotica e dispositivi medicali.

24.3 Idea chiave

La differenza fondamentale tra soft e hard real-time è la **rigorosità della scadenza**: nei sistemi soft è consigliabile rispettarla, nei sistemi hard è obbligatorio. Lo scheduler deve scegliere priorità e tempi di esecuzione in funzione del tipo di real-time richiesto.

24.4 Osservazioni

- Nei sistemi soft real-time, la flessibilità permette un migliore utilizzo delle CPU condivise, riducendo overhead e congestione.
- Nei sistemi hard real-time, l'overhead della schedulazione deve essere minimo, poiché ogni ritardo può compromettere l'efficacia del sistema.
- La progettazione di schedulatori real-time richiede considerazione di:
  - Priorità statiche vs dinamiche
  - Preemption dei task a bassa priorità
  - Predictibilità dei tempi di esecuzione
- La scelta tra soft e hard real-time dipende dall'applicazione e dal costo accettabile dei ritardi.

25 Event Latency nei Sistemi Real-Time

25.1 Introduzione

I sistemi real-time sono **guidati da eventi**: l'esecuzione dei task dipende dalla comparsa di eventi esterni o interni. Un concetto fondamentale in questo contesto è la **latency**, cioè il ritardo tra il momento in cui un evento si verifica e il momento in cui il sistema risponde.

25.2 Definizione di Event Latency

- **Event latency** = intervallo di tempo tra il verificarsi di un evento e la risposta effettiva del sistema.
- Questa metrica è critica per garantire che i task real-time rispettino le scadenze e siano efficaci.

25.3 Requisiti di latenza in diversi contesti

La tolleranza alla latenza dipende dall'applicazione del sistema real-time:

- **ABS (Anti-lock Braking System)**: richiede latenza molto bassa, tipicamente **3–5 ms**, per garantire sicurezza e reattività immediata.
- **Radar aeronautico**: può tollerare latenze dell'ordine di secondi, poiché la velocità degli eventi e la criticità immediata sono inferiori rispetto ai sistemi di controllo veicolare.

25.4 Idea chiave

Nei sistemi real-time, la latenza è un parametro fondamentale che influenza la progettazione dello scheduler e l'architettura del sistema. Task critici richiedono latenze minime prevedibili, mentre task meno critici possono tollerare ritardi maggiori.

25.5 Osservazioni

- La latenza dipende da:
  - Overhead dello scheduler
  - Preemption dei task in corso
  - Accesso a risorse condivise
  - Eventuali stall della memoria o della cache
- Nei sistemi real-time hard, ogni componente hardware e software deve essere progettato per minimizzare e rendere prevedibile la latenza.
- Nei sistemi soft real-time, la latenza può essere più flessibile, permettendo una migliore utilizzazione della CPU e delle risorse.
- Misurare e rispettare i vincoli di latenza è fondamentale per garantire l'affidabilità e la sicurezza delle applicazioni real-time critiche.

26 Latency nei Sistemi Real-Time

26.1 Tipi principali di latenza

Nei sistemi real-time, la latenza può essere scomposta in due tipi principali:

1. **Interrupt Latency** (latenza di interruzione)
2. **Dispatch Latency** (latenza di dispatch)

26.2 Interrupt Latency

La **latenza di interruzione** è il tempo che intercorre tra l'arrivo di un interrupt e l'inizio dell'esecuzione della *Interrupt Service Routine (ISR)* associata. Questa latenza dipende dalle operazioni necessarie per gestire correttamente il contesto corrente e iniziare l'ISR:

1. Terminare l'esecuzione dell'istruzione corrente (alcune istruzioni non sono interrompibili a livello hardware).
2. Identificare l'interrupt ricevuto, selezionando la routine corretta.
3. Salvare lo stato del processo in esecuzione (*context save*), in modo che possa riprendere dopo l'ISR.
4. Avviare l'esecuzione della ISR.

26.3 Idea chiave

La **interrupt latency** definisce quanto rapidamente un sistema real-time può reagire a eventi esterni. Minore è questa latenza, più il sistema è reattivo e adatto a task critici. Lo scheduler e l'architettura hardware devono minimizzare questo tempo per rispettare le scadenze dei task hard real-time.

26.4 Osservazioni

- L'interrupt latency dipende da fattori hardware (pipeline, priorità interrupt, bus) e software (overhead dello scheduler, gestione dello stack e salvataggio del contesto).
- Nei sistemi hard real-time, l'interrupt latency deve essere **deterministica** e prevedibile.
- In sistemi soft real-time, una latenza non perfetta è tollerabile, ma influisce sulle performance complessive e sulla reattività percepita.
- Ottimizzazioni possibili includono:
  - Ridurre il numero di istruzioni critiche non interrompibili
  - Gestione hardware delle priorità degli interrupt
  - Pre-salvataggio di contesti comuni per ISR frequenti

27 Interrupt e Dispatch Latency nei Sistemi Real-Time

27.1 Introduzione

Nei sistemi real-time, la reattività ai task critici dipende fortemente dalla gestione della **latency**. Due componenti principali definiscono la velocità di risposta del sistema:

1. **Interrupt latency** (latenza di interruzione)
2. **Dispatch latency** (latenza di dispatch)

27.2 Riduzione della Interrupt Latency

- Nei sistemi real-time, è essenziale **minimizzare l'interrupt latency**.
- Nei sistemi **hard real-time**, l'interrupt latency deve rimanere **al di sotto di un limite massimo rigoroso** per garantire il rispetto delle scadenze dei task critici.
- La latenza dipende da:
  - Overhead hardware e pipeline della CPU
  - Priorità degli interrupt
  - Salvataggio e ripristino del contesto del processo corrente

27.3 Dispatch Latency

- La **dispatch latency** è il tempo necessario per fermare un processo in esecuzione e iniziare un altro processo pronto.
- Per i task real-time, la dispatch latency deve essere **minimizzata** per garantire accesso immediato alla CPU.
- La strategia più efficace è l'uso di un **kernel preemptive**, che può sospendere rapidamente i processi a bassa priorità per eseguire task critici.

27.4 Misurazioni nei sistemi hard real-time

- Nei sistemi hard real-time, sia l'interrupt latency sia la dispatch latency vengono misurate in **microsecondi** (μs) per assicurare che le scadenze temporali siano rispettate.
- Una latenza prevedibile e bassa è cruciale per applicazioni come controllo industriale, avionica e sistemi automotive.

27.5 Idea chiave

Per garantire il corretto funzionamento di sistemi real-time hard, è necessario:

- Minimizzare sia l'interrupt che la dispatch latency
- Garantire valori prevedibili e deterministici
- Utilizzare kernel preemptive e ottimizzazioni hardware per ISR

27.6 Osservazioni

- La differenza tra soft e hard real-time si manifesta principalmente nei vincoli su queste latenze: soft real-time può tollerare occasionali ritardi, hard real-time no.
- La gestione efficace della dispatch latency consente al sistema di rispettare le priorità dei task e di fornire risposta immediata agli eventi critici.
- Tecniche di ottimizzazione includono:
  - Preemption rapida dei processi a bassa priorità
  - ISR ottimizzate e snelle
  - Riduzione dei lock e overhead software nelle sezioni critiche

28 Schedulazione Hard Real-Time

28.1 Introduzione

Nei sistemi **real-time hard**, il sistema operativo deve rispondere immediatamente ai task critici, garantendo il rispetto delle scadenze temporali. A differenza dei sistemi soft real-time, dove occasionali ritardi possono essere tollerati, nei sistemi hard real-time i task devono essere completati entro vincoli rigorosi.

28.2 Politica di Scheduling

- I sistemi hard real-time utilizzano schedulazione **preemptive basata su priorità**.
- Task critici a priorità più alta possono interrompere l'esecuzione di task a priorità inferiore, garantendo risposta immediata.
- La schedulazione soft real-time è garantita solo come effetto collaterale a questo livello.

28.3 Caratteristiche dei Task Periodici

I task periodici in hard real-time sono definiti da tre parametri principali:

1. **Tempo di esecuzione** (t): tempo necessario al processore per completare il task.
2. **Deadline** (d): momento massimo entro cui il task deve terminare.
3. **Periodo** (p): intervallo di tempo tra due attivazioni successive del task.

Questi parametri devono soddisfare la condizione:

$$0 \leq t \leq d \leq p$$

28.4 Admission Control

- Lo scheduler implementa un **controllo di ammissione**, accettando un nuovo task solo se è possibile garantire che tutte le scadenze saranno rispettate.
- Questo meccanismo evita sovraccarichi del sistema che comprometterebbero la capacità di rispettare i vincoli dei task hard real-time.

28.5 Idea chiave

La schedulazione hard real-time si concentra sulla **garanzia deterministica** che i task critici vengano completati entro le scadenze. L'utilizzo di priorità, preemption e admission control è fondamentale per assicurare affidabilità e correttezza temporale.

28.6 Osservazioni

- L'analisi dei task periodici permette allo scheduler di determinare se il sistema può sostenere un carico aggiuntivo senza violare scadenze.
- I task a priorità inferiore possono essere preemptati frequentemente, quindi la progettazione di questi task deve considerare latenze e interruzioni possibili.
- Questo tipo di schedulazione è essenziale in applicazioni critiche come:
  - Sistemi di controllo industriale
  - Avionica e aerospaziale
  - Automotive (es. ABS, controllo motore)
  - Dispositivi medicali
- La combinazione di preemption, priorità e admission control garantisce che i sistemi hard real-time siano affidabili e prevedibili.

29 Schedulazione Rate-Monotonic (RMS)

29.1 Introduzione

Il **Rate-Monotonic Scheduling (RMS)** è uno degli algoritmi fondamentali di schedulazione **hard real-time**. Si basa su priorità statiche e preemption, ed è particolarmente indicato per task periodici.

29.2 Principio di funzionamento

- Ogni task periodico riceve una **priorità statica** determinata dal suo **periodo** p:
  - **Periodo più breve** $\rightarrow$ priorità più alta
  - **Periodo più lungo** $\rightarrow$ priorità più bassa
- L'idea è che i task che devono essere eseguiti più frequentemente abbiano priorità maggiore per garantire che nessuna scadenza venga violata.

29.3 Assunzioni RMS

- Ogni task ha un **tempo di esecuzione fisso** per ogni burst CPU, cioè il tempo di calcolo richiesto non varia.
- I task sono periodici e indipendenti.
- Lo scheduler è **preemptive**: un task a priorità più alta può sospendere temporaneamente un task a priorità inferiore.

29.4 Idea chiave

RMS sfrutta la conoscenza dei periodi dei task per assegnare priorità statiche: i task con periodi più brevi, che richiedono esecuzioni più frequenti, ottengono priorità più alta. Questo approccio consente di soddisfare scadenze in sistemi hard real-time con task periodici.

29.5 Osservazioni

- RMS è semplice da implementare e analizzare, ma ottimale solo sotto le assunzioni standard (task periodici, tempo di esecuzione fisso, indipendenza dei task).
- In presenza di carichi elevati, il sistema può raggiungere un **utilizzo massimo teorico** di circa 69% per un numero arbitrario di task periodici, oltre il quale non si può garantire la schedulazione senza violare scadenze.
- RMS è adatto a sistemi embedded e industriali, dove task periodici devono rispettare vincoli temporali rigorosi.
- La combinazione con kernel preemptive e monitoraggio della latenza assicura che le scadenze vengano rispettate anche in presenza di task a priorità inferiore.

30 Verifica di Fattibilità con Due Task Periodici

30.1 Introduzione

Prima di schedulare task hard real-time, è fondamentale verificare se le scadenze possono essere rispettate. Questo esempio mostra come una scelta non corretta delle priorità possa portare a **missed deadlines** anche se l'utilizzo totale della CPU è inferiore a 100%.

30.2 Dati del problema

Consideriamo due task periodici:

- P_1: periodo $p_1 = 50\,\text{ms}$, tempo di esecuzione $t_1 = 20\,\text{ms}$
- P_2: periodo $p_2 = 100\,\text{ms}$, tempo di esecuzione $t_2 = 35\,\text{ms}$
- Le **deadline** coincidono con l'inizio del periodo successivo:

$$\text{deadline}_i = \text{inizio del prossimo periodo}_i$$

30.3 Calcolo dell'utilizzo della CPU

L'utilizzo di ciascun task è dato da:

$$U_1 = \frac{t_1}{p_1} = \frac{20}{50} = 0.40$$

$$U_2 = \frac{t_2}{p_2} = \frac{35}{100} = 0.35$$

$$U_{\text{totale}} = U_1 + U_2 = 0.75$$

Nota: l'utilizzo totale è inferiore a 1, ma questo non garantisce che tutte le scadenze siano rispettate senza una corretta assegnazione delle priorità.

30.4 Assegnazione di priorità non ottimale

Supponiamo una scelta **naive**:

- P_2 ha priorità più alta di P_1

30.5 Esecuzione prevista

- P_2 inizia al tempo 0 ms e termina a 35 ms
- P_1 inizia subito dopo, alle 35 ms, e termina alle 55 ms

30.6 Risultato

- La scadenza di P_1 era al tempo 50 ms
- Il completamento effettivo è avvenuto a 55 ms → **missed deadline**

30.7 Spiegazione

- Nonostante l'utilizzo totale sia 0.75, l'assegnazione di priorità errata ha fatto sì che un task con periodo più breve (P_1) fosse ritardato da un task con periodo più lungo (P_2).
- Questo esempio mostra l'importanza di utilizzare algoritmi come **Rate-Monotonic Scheduling (RMS)**, dove i task con periodi più brevi hanno priorità più alta, garantendo la correttezza delle scadenze.

30.8 Idea chiave

Per task periodici in sistemi hard real-time, la scelta corretta delle priorità è critica. Anche un utilizzo totale inferiore a 1 non garantisce scadenze rispettate se i task con periodi brevi non hanno priorità sufficientemente elevate.

31 Verifica di Fattibilità con Priorità Corretta per Due Task Periodici

31.1 Introduzione

Nel caso precedente, abbiamo visto come un'assegnazione di priorità errata può portare a **missed deadlines**. Ora consideriamo una strategia corretta secondo il principio di **Rate-Monotonic Scheduling (RMS)**, dove il task con periodo più breve ottiene priorità più alta.

31.2 Assegnazione di priorità corretta

- P_1 (periodo più breve, $p_1 = 50\,\text{ms}$) ottiene priorità più alta
- P_2 (periodo più lungo, $p_2 = 100\,\text{ms}$) ottiene priorità più bassa

31.3 Esecuzione prevista con preemption

1. **Prima iterazione:**
   - P_1 inizia al tempo 0 ms $\rightarrow$ termina a 20 ms
   - P_2 inizia a 20 ms $\rightarrow$ esegue fino a 50 ms (ha bisogno di 35 ms, quindi mancano ancora 15 ms)
2. **Preemption:**
   - Al tempo 50 ms, un nuovo periodo di P_1 inizia
   - P_2 viene preempted e P_1 ottiene la CPU
   - P_1 termina il suo secondo burst a 70 ms
3. **Completamento di P_2:**
   - P_2 riprende l'esecuzione a 70 ms $\rightarrow$ termina a 75 ms

31.4 Risultato

- Entrambi i task rispettano la loro prima scadenza:
  - P_1 completato entro 50 ms
  - P_2 completato entro 100 ms
- La CPU rimane inattiva da 75 ms a 100 ms, in attesa dell'inizio del prossimo periodo di P_1

31.5 Spiegazione dettagliata

- L'uso di **priorità statiche basate sul periodo** permette di rispettare le scadenze anche quando i task si sovrappongono.
- La **preemption** è fondamentale: il task con priorità più alta può sospendere temporaneamente l'esecuzione di un task a priorità inferiore, garantendo che il task critico inizi immediatamente.
- L'intervallo di CPU idle tra 75 ms e 100 ms è normale e non indica inefficienza: il sistema è progettato per rispettare le scadenze dei task periodici senza overbooking della CPU.

31.6 Idea chiave

Assegnare priorità in base ai periodi dei task (più breve → più alta priorità) e utilizzare un kernel preemptive consente di rispettare tutte le scadenze in sistemi hard real-time con task periodici.

31.7 Osservazioni

- Questo esempio illustra l'efficacia di RMS rispetto a una priorità naive.
- La preemption consente di gestire correttamente task con periodi differenti, evitando ritardi critici.
- La CPU idle periodica è prevista e consente di gestire i task successivi senza violare scadenze future.

32 Limitazioni della Schedulazione a Priorità Basata sulla Frequenza

32.1 Introduzione

La schedulazione a priorità statica basata sulla frequenza di esecuzione, tipica del **Rate-Monotonic Scheduling (RMS)**, assegna priorità più alta ai task con periodo più breve (più frequenti). Tuttavia, questo approccio presenta limitazioni quando il carico del sistema è elevato.

32.2 Principio di ottimalità

- La schedulazione a priorità proporzionale alla frequenza è **ottimale per assegnazioni statiche di priorità** sotto le assunzioni standard (task periodici, indipendenti, tempo di esecuzione fisso).
- Se questo metodo non riesce a rispettare le scadenze di un task set, nessun altro schema di priorità statica può farlo.

32.3 Esempio pratico

Consideriamo due task:

- P_1: periodo $p_1 = 50\,\text{ms}$, tempo di esecuzione $t_1 = 25\,\text{ms}$
- P_2: periodo $p_2 = 80\,\text{ms}$, tempo di esecuzione $t_2 = 35\,\text{ms}$

32.4 Calcolo dell'utilizzo della CPU

$$U_{\text{totale}} = \frac{t_1}{p_1} + \frac{t_2}{p_2} = \frac{25}{50} + \frac{35}{80} = 0.50 + 0.4375 = 0.9375 \approx 0.94$$

32.5 Analisi

- L'utilizzo totale della CPU è circa il 94%, molto vicino alla saturazione.
- Anche assegnando priorità in base al periodo (P più alta priorità), il task set non può essere schedulato senza violare le scadenze.
- Questo dimostra un **limite teorico**: esiste un utilizzo massimo oltre il quale i task periodici non sono garantiti di rispettare le scadenze con priorità statica.

32.6 Conclusione

- La schedulazione a priorità statica basata sulla frequenza non può garantire scadenze per task set con utilizzo elevato (tipicamente superiore al limite teorico RMS: circa 69% per un numero arbitrario di task).
- In questi casi, algoritmi dinamici come **Earliest-Deadline-First (EDF)** possono schedulare il task set con successo, sfruttando l'utilizzo massimo della CPU fino al 100%.

32.7 Idea chiave

La frequenza di esecuzione come criterio di priorità funziona bene per carichi moderati, ma presenta limiti intrinseci quando il sistema è vicino alla saturazione. Per garantire la schedulazione di tutti i task in condizioni di carico elevato, è necessario considerare algoritmi a priorità dinamica come EDF.

33 Limite di Utilizzo della CPU nella Schedulazione a Priorità Basata sulla Frequenza

33.1 Introduzione

La schedulazione a priorità statica basata sulla frequenza, tipica del **Rate-Monotonic Scheduling (RMS)**, è ottimale tra gli algoritmi di priorità fissa. Tuttavia, presenta un **limite teorico di utilizzo della CPU**, oltre il quale non si può garantire che tutti i task rispettino le scadenze.

33.2 Limite massimo di utilizzo per N task

Per un insieme di N task periodici, il **peggior caso** di utilizzo massimo della CPU è dato dalla formula:

$$U_{\max} = N\left(2^{1/N} - 1\right)$$

Dove:

- N = numero totale di task periodici
- $U_{\max}$ = utilizzo massimo della CPU per cui RMS garantisce tutte le scadenze

33.3 Esempi pratici

- **Un solo task (N=1):**

$$U_{\max} = 1 \cdot (2^{1/1} - 1) = 1 \cdot (2 - 1) = 1 \Rightarrow 100\% \text{ della CPU}$$

- **Due task (N=2):**

$$U_{\max} = 2 \cdot (2^{1/2} - 1) = 2 \cdot (1.4142 - 1) \approx 0.8284 \Rightarrow 83\% \text{ della CPU}$$

- **Molti task (N $\to \infty$):**

$$U_{\max} \to \ln 2 \approx 0.693 \Rightarrow 69\% \text{ della CPU}$$

33.4 Spiegazione

- La formula deriva dall'analisi di schedulabilità di task periodici con priorità fissa (Liu & Layland, 1973).
- Il limite indica che, anche in condizioni ottimali, RMS non può garantire le scadenze se il carico totale supera $U_{\max}$.
- Con pochi task, il limite è relativamente alto (fino al 100% per un solo task), ma man mano che il numero di task aumenta, il limite si stabilizza attorno al 69%.

33.5 Idea chiave

- La schedulazione a priorità fissa è **ottimale** tra gli algoritmi statici, ma ha un **vincolo intrinseco di utilizzo della CPU**.
- Per sistemi con elevato numero di task o alto utilizzo della CPU, è necessario considerare algoritmi a priorità **dinamica** come **Earliest-Deadline-First (EDF)**, che possono sfruttare fino al 100% della CPU mantenendo la schedulabilità.

33.6 Osservazioni

- Il limite di utilizzo non implica inefficienza: indica solo la massima capacità garantita dal punto di vista teorico.
- La conoscenza di questo limite è essenziale per progettare sistemi hard real-time affidabili.
- RMS rimane semplice da implementare e analizzare, ma deve essere usato con attenzione in scenari con numerosi task o carichi prossimi alla saturazione.

34 Schedulazione Earliest-Deadline-First (EDF)

34.1 Introduzione

Earliest-Deadline-First (EDF) è un algoritmo di schedulazione **dinamica** per sistemi real-time. A differenza degli schemi a priorità fissa come RMS, EDF assegna le priorità dei task in base alle loro **scadenze imminenti**:

- **Deadline più vicina** → priorità più alta
- Ogni task deve comunicare al sistema la propria **prossima scadenza** quando è pronto per l'esecuzione

EDF è teoricamente ottimale: se un insieme di task periodici può essere schedulato senza violare scadenze, EDF lo farà.

34.2 Esempio pratico

Consideriamo due task periodici:

- P_1: periodo $p_1 = 50\,\text{ms}$, tempo di esecuzione $t_1 = 25\,\text{ms}$
- P_2: periodo $p_2 = 80\,\text{ms}$, tempo di esecuzione $t_2 = 35\,\text{ms}$

34.3 Esecuzione con EDF

1. **Prima iterazione:**
   - P_1 inizia al tempo 0 ms → termina a 25 ms
   - P_2 inizia subito dopo, eseguendo fino a 50 ms (la scadenza di P è 80 ms, quindi non ha priorità più alta rispetto a P)
2. **Continua esecuzione di P:**
   - P_2 prosegue fino a completamento alle 60 ms circa
   - Entrambi i task rispettano la prima scadenza (P_1 50 ms, P_2 80 ms)
3. **Secondo ciclo (al tempo 100 ms):**
   - P_1 ha la scadenza più vicina → preemption di P_2
   - P_1 esegue per 25 ms
   - P_2 riprende dopo → tutte le scadenze rispettate

34.4 Vantaggi di EDF

- **Teoricamente ottimale**: se un task set può essere schedulato, EDF garantisce che tutte le scadenze siano rispettate.
- Consente di sfruttare fino al **100% della CPU**, a differenza di RMS che ha un limite teorico ($\approx 69\%$ per molti task).
- Adatto a sistemi con carichi variabili o task con periodi diversi.

34.5 Limitazioni pratiche

- Performance reali possono degradare a causa di:
  - Overhead del kernel per il calcolo dinamico delle priorità
  - Preemption frequente tra task
  - Ritardi dovuti a lock, ISR o latenza di dispatch
- EDF richiede che ogni task comunichi le proprie scadenze, aumentando la complessità di implementazione rispetto a RMS.

34.6 Idea chiave

EDF è l'algoritmo di schedulazione più flessibile per task periodici in sistemi hard real-time, ottimizzando l'utilizzo della CPU e garantendo la correttezza delle scadenze, purché il sistema gestisca correttamente preemption e overhead.

35 Schedulazione a Quote Proporzionali (Proportional Share Scheduling)

35.1 Introduzione

La **Schedulazione a Quote Proporzionali** è una tecnica di allocazione della CPU basata su **parti (shares)** di tempo processore. L'idea principale è garantire a ogni processo una frazione determinata di CPU, proporzionale al numero di quote assegnate.

35.2 Principio di funzionamento

- La CPU viene divisa in T **quote totali**.
- Ogni processo riceve N quote e ottiene così una frazione N/T del tempo CPU.
- La schedulazione garantisce che il processo non superi la percentuale di CPU assegnata.

35.3 Esempio pratico

Consideriamo un sistema con:

- Quote totali: $T = 100$
- Processo $A \rightarrow 50$ quote $\rightarrow 50\%$ CPU
- Processo $B \rightarrow 15$ quote $\rightarrow 15\%$ CPU
- Processo $C \rightarrow 20$ quote $\rightarrow 20\%$ CPU
- Quote rimanenti: 15 non assegnate

35.4 Gestione tramite Admission Control

- Il sistema verifica che siano disponibili quote sufficienti prima di ammettere un nuovo processo.
- Un processo viene ammesso solo se esiste una quantità sufficiente di quote per garantirne la percentuale di CPU desiderata.
- Questo meccanismo impedisce che la somma delle percentuali superi il 100% della CPU, assicurando garanzie temporali precise.

35.5 Idea chiave

La schedulazione a quote proporzionali permette una **divisione prevedibile e garantita del tempo CPU**, utile in sistemi con multipli processi concorrenti che richiedono risorse garantite. È particolarmente adatta a:

- Sistemi interattivi con risorse condivise
- Ambienti multiutente
- Sistemi real-time soft, dove si desidera equilibrio tra task concorrenti

35.6 Osservazioni

- La percentuale di CPU assegnata a ogni processo rimane fissa finché non cambia l'assegnazione delle quote.
- L'uso di admission control garantisce stabilità e prevedibilità, evitando overbooking della CPU.
- Le quote non assegnate possono essere distribuite dinamicamente a processi esistenti in sistemi più avanzati, introducendo concetti di fair-share scheduling.

36 Esempio di Valutazione Analitica degli Algoritmi di Schedulazione

36.1 Introduzione

La **valutazione analitica** degli algoritmi di schedulazione studia le prestazioni dei diversi schemi per un carico di lavoro definito, utilizzando modelli deterministici:

- L'approccio deterministico assume un insieme fisso di processi con tempi di burst noti.
- Permette di confrontare algoritmi come FCFS, SJF, Round Robin e altri in termini di metriche come **tempo medio di attesa**, **turnaround time** e **throughput**.

36.2 Workload considerato

Consideriamo un esempio con cinque processi, tutti arrivati al tempo 0, con burst CPU specifici (in ms):

Processo	CPU Burst (ms)
P_1	24
P_2	3
P_3	3
P_4	8
P_5	15

36.3 Confronto tra algoritmi

- **First-Come, First-Served (FCFS)**
  - I processi vengono eseguiti nell'ordine di arrivo.
  - **Tempo medio di attesa**: 28 ms
  - Limite principale: convoglio di processi lunghi aumenta il tempo di attesa dei processi brevi.
- **Shortest Job First (SJF) non preemptive**
  - I processi con burst più breve vengono eseguiti prima.
  - **Tempo medio di attesa**: 13 ms
  - Risultato migliore per questo workload, poiché minimizza l'attesa media dei processi brevi.
- **Round Robin (quantum = 10 ms)**
  - Ogni processo ottiene una fetta di CPU di 10 ms, poi passa in coda.
  - **Tempo medio di attesa**: 23 ms
  - Limite: overhead dovuto ai context switch e tempi di attesa maggiori per processi brevi.

36.4 Conclusione

- Per il carico di lavoro deterministico considerato, il miglior algoritmo è **SJF non preemptive** con un tempo medio di attesa di 13 ms.
- La valutazione analitica permette di:
  - Confrontare prestazioni teoriche di diversi schemi di schedulazione
  - Scegliere l'algoritmo più adatto in base al tipo di workload
  - Evidenziare trade-off tra complessità, preemption e tempi medi di attesa

36.5 Idea chiave

La valutazione analitica è utile per **dimensionare e progettare sistemi operativi e schedulatori**, fornendo un riferimento teorico sulle prestazioni prima di test su carichi dinamici reali.

37 Valutazione della Schedulazione tramite Simulazione

37.1 Introduzione

La simulazione è un metodo pratico per valutare le prestazioni degli algoritmi di schedulazione in scenari realistici, soprattutto quando il comportamento dei processi non è deterministico o dipende da eventi esterni.

37.2 Principi generali

- Si modella il sistema operativo e le risorse della CPU.
- Un **orologio virtuale** aggiorna continuamente lo stato dei processi e della CPU.
- Eventi come arrivi di processi, completamenti di CPU burst o operazioni I/O vengono generati durante la simulazione.

37.3 Metodi di simulazione

1. **Simulazione basata su numeri casuali (probabilistica)**
   - Gli eventi vengono generati utilizzando generatori di numeri casuali.
   - Utile per analizzare scenari generali e comportamenti medi.
   - Limite: l'ordine reale degli eventi può non essere rispettato, producendo risultati meno precisi per casi specifici.
2. **Simulazione trace-driven (basata su tracce)**
   - Utilizza sequenze di eventi reali registrate da un sistema operativo.
   - Fornisce un confronto più accurato degli algoritmi di schedulazione, poiché l'ordine effettivo degli eventi viene preservato.
   - Ideale per valutare performance realistiche in scenari complessi.

37.4 Vantaggi della simulazione

- Permette di confrontare diversi algoritmi di schedulazione prima di implementazioni reali.
- Aiuta a identificare overhead, congestione della CPU e tempi medi di attesa in scenari complessi.
- Consente di testare sistemi con carichi variabili e condizioni estreme senza rischiare instabilità del sistema reale.

37.5 Idea chiave

La simulazione rappresenta uno strumento fondamentale per **valutare l'efficacia degli algoritmi di schedulazione** in scenari realistici, bilanciando accuratezza, complessità e riproducibilità dei risultati.

38 Il Problema della Sezione Critica

38.1 Introduzione

Nei sistemi operativi, i processi possono **cooperare** condividendo dati o risorse comuni. Questa cooperazione, se non gestita correttamente, può causare **inconsistenze** nei dati condivisi o comportamenti imprevedibili dei processi.

38.2 Sezione critica

- Ogni processo può avere una **sezione critica** (*critical section*), ovvero il blocco di codice in cui vengono **modificati dati condivisi**.
- La regola fondamentale: **solo un processo alla volta può eseguire la sua sezione critica**.
- L'accesso simultaneo alla sezione critica da più processi può causare:
  - Dati incoerenti
  - Race conditions
  - Errori di sincronizzazione

38.3 Protocollo per la cooperazione

Per garantire l'accesso corretto alla sezione critica, è necessario definire un **protocollo di sincronizzazione** che stabilisca le regole di accesso per i processi cooperanti.

38.4 Struttura di un processo cooperante

Un processo che utilizza sezioni critiche può essere concettualmente diviso in quattro parti:

$$\underbrace{\text{Entry Section}}_{\text{sezione di entrata}} \rightarrow \underbrace{\text{Critical Section}}_{\text{sezione critica}} \rightarrow \underbrace{\text{Exit Section}}_{\text{sezione di uscita}} \rightarrow \underbrace{\text{Remainder Section}}_{\text{sezione restante}}$$

- **Entry Section**: codice che prepara l'accesso alla sezione critica e controlla che nessun altro processo stia eseguendo la propria sezione critica.
- **Critical Section**: codice che accede o modifica dati condivisi.
- **Exit Section**: codice che segnala che il processo ha completato l'uso della sezione critica, permettendo ad altri processi di entrare.
- **Remainder Section**: codice che non accede a risorse condivise, può essere eseguito senza restrizioni di sincronizzazione.

38.5 Idea chiave

Il **problema della sezione critica** è fondamentale nella sincronizzazione dei processi: senza un protocollo appropriato, i processi cooperanti possono interferire tra loro, causando inconsistenze nei dati condivisi e comportamenti imprevedibili del sistema operativo.

39 Requisiti per le Soluzioni della Sezione Critica

39.1 Introduzione

Per risolvere correttamente il problema della sezione critica, un protocollo di sincronizzazione deve soddisfare tre requisiti fondamentali. Questi requisiti garantiscono che i processi cooperanti possano accedere in modo sicuro ai dati condivisi senza causare deadlock o starvation.

39.2 1. Mutua esclusione (Mutual Exclusion)

- Definizione: **nessun due processi devono trovarsi nella sezione critica contemporaneamente**.
- Spiegazione: questo requisito previene **race conditions** e garantisce che le modifiche ai dati condivisi siano consistenti.
- Implicazioni pratiche: un protocollo che non rispetta la mutua esclusione non può essere considerato corretto, poiché l'integrità dei dati è compromessa.

39.3 2. Progress (Progresso)

- Definizione: se nessun processo si trova nella sezione critica e alcuni processi vogliono entrare, la scelta di chi entra successivamente non può essere **indefinitamente ritardata**.
- Spiegazione: garantisce che il sistema continui a funzionare e che i processi non rimangano bloccati senza motivo.
- Implicazioni pratiche: il protocollo deve permettere ai processi di prendere decisioni e avanzare senza deadlock o attese infinite.

39.4 3. Attesa limitata (Bounded Waiting)

- Definizione: ogni processo che desidera entrare nella sezione critica deve avere un numero **limitato di tentativi di accesso da parte di altri processi** prima di poter eseguire la propria sezione critica.
- Spiegazione: questo requisito previene la **starvation**, cioè l'attesa indefinita di un processo.
- Implicazioni pratiche: il protocollo deve garantire che ogni processo ottenga l'accesso alla sezione critica entro un numero finito di turni.

39.5 Idea chiave

Un protocollo di sincronizzazione corretto per le sezioni critiche deve garantire:

1. **Mutua esclusione**: protezione dei dati condivisi
2. **Progresso**: nessun blocco indefinito dei processi
3. **Attesa limitata**: ogni processo ottiene accesso entro un numero finito di tentativi

Questi tre requisiti costituiscono la base per tutti gli algoritmi successivi di sincronizzazione dei processi.

40 Sezioni Critiche e Progettazione del Kernel

40.1 Introduzione

La protezione delle sezioni critiche non riguarda solo i processi utente, ma anche il **kernel** del sistema operativo, che gestisce risorse condivise critiche. La progettazione del kernel deve quindi garantire che le sezioni critiche interne siano sicure e consistenti.

40.2 Single-Core Systems

- Nei sistemi con un singolo core, il metodo più semplice per proteggere una sezione critica è **disabilitare gli interrupt**.
- Disabilitando gli interrupt, il kernel garantisce che nessun altro processo o ISR (Interrupt Service Routine) possa interrompere l'esecuzione del codice critico.
- Questa soluzione è efficace su sistemi singolo-core perché la CPU esegue un solo thread alla volta.

40.3 Multiprocessor Systems

- Nei sistemi multiprocessore, disabilitare gli interrupt diventa **inefficiente e troppo lento**, perché:
  - Altri core continuerebbero a eseguire codice potenzialmente critico
  - La gestione globale degli interrupt è complessa
- Perciò, sono necessarie strategie più sofisticate.

40.4 Strategie del Kernel

Esistono due principali strategie per la gestione della sezione critica nel kernel:

1. **Kernel preemptive (preemptive kernel)**
   - I processi in modalità kernel possono essere **preempted**, cioè sospesi da altri processi più prioritari.
   - Vantaggio: **migliore reattività** del sistema e supporto per task real-time.
   - Svantaggio: il kernel deve proteggere accuratamente tutte le **strutture dati condivise** per evitare race conditions.
2. **Kernel non-preemptive (nonpreemptive kernel)**
   - I processi in modalità kernel non possono essere preempted.
   - Vantaggio: elimina automaticamente alcune race conditions nelle sezioni critiche del kernel.
   - Svantaggio: peggiora la **responsività**, poiché un processo kernel lungo può bloccare altri processi.

40.5 Preferenze pratiche

- I kernel preemptive sono generalmente preferiti nei sistemi moderni, grazie a:
  - Migliore reattività e supporto per task real-time
  - Possibilità di eseguire task ad alta priorità senza attese lunghe
- Tuttavia, richiedono un'implementazione attenta dei meccanismi di protezione delle sezioni critiche del kernel.

40.6 Idea chiave

La gestione delle sezioni critiche nel kernel dipende dall'architettura hardware e dalle esigenze del sistema:

- Single-core: disabilitare gli interrupt è semplice e sufficiente
- Multiprocessor: kernel preemptive con protezione accurata dei dati è preferibile
- La scelta tra kernel preemptive e non-preemptive bilancia **responsività** e **semplicità della protezione dati**.

41 Soluzione di Peterson per il Problema della Sezione Critica

41.1 Introduzione

La **Soluzione di Peterson** è un algoritmo software classico che risolve il problema della sezione critica per due processi cooperanti, garantendo:

- **Mutua esclusione**: nessun processo entra contemporaneamente nella sezione critica
- **Progresso**: se nessun processo è nella sezione critica, un processo in attesa può entrare senza ritardi indefiniti
- **Attesa limitata**: nessun processo subisce starvation

L'algoritmo funziona esclusivamente per **due processi**, indicati come P_0 e P_1.

41.2 Variabili condivise

L'algoritmo utilizza le seguenti variabili condivise:

- `boolean flag[2]`: segnala l'intento di ciascun processo di entrare nella sezione critica.
- `int turn`: indica di chi è il turno, cioè quale processo ha la precedenza se entrambi vogliono accedere contemporaneamente.

41.3 Idea principale

- `flag[i]` = `true` significa che il processo P_i desidera entrare nella sezione critica.
- `turn` $= j$ indica che, in caso di conflitto, si dà la priorità al processo P_j.
- Se entrambi i processi cercano di entrare contemporaneamente, il valore finale di `turn` decide quale dei due accederà per primo alla sezione critica.

41.4 Funzionamento passo passo

Quando un processo P_i vuole entrare nella sezione critica:

1. **Segnalazione dell'intento:** P_i imposta `flag[i]` = `true`.
2. **Concessione del turno all'altro processo:** P_i imposta `turn` $= j$, dando così a P_j la possibilità di entrare prima nella sezione critica.
3. **Attesa attiva (busy wait):** P_i entra in un ciclo di attesa finché (`flag[j]` $\wedge$ `turn` $= j$) è vero.
   - Se P_j non vuole entrare (`flag[j]` = `false`), P_i può accedere subito.
   - Se P_j vuole entrare, il valore di `turn` decide chi entra per primo.

41.5 Codice in pseudolinguaggio

```
flag[0] = flag[1] = false;
turn = 0;

process P0() {
    while (true) {
        flag[0] = true;
        turn = 1;
        while (flag[1] && turn == 1); // attesa attiva
        // Sezione critica
        ...
        flag[0] = false;
        // Sezione restante
        ...
    }
}

process P1() {
    while (true) {
        flag[1] = true;
        turn = 0;
        while (flag[0] && turn == 0); // attesa attiva
        // Sezione critica
        ...
        flag[1] = false;
        // Sezione restante
        ...
    }
}
```

41.6 Spiegazione dettagliata del funzionamento

- Ogni processo segnala il proprio intento e concede all'altro processo la precedenza.
- Se entrambi tentano di entrare contemporaneamente, solo un valore finale di `turn` sopravvive, determinando chi entra per primo nella sezione critica.
- Una volta terminata la sezione critica, il processo resetta `flag[i] = false`, permettendo all'altro processo di entrare.
- Questo meccanismo semplice garantisce **mutua esclusione, progress e attesa limitata** senza utilizzare hardware speciale.

41.7 Idea chiave

La Soluzione di Peterson dimostra che è possibile risolvere il problema della sezione critica con un semplice protocollo software per due processi, sfruttando solo variabili condivise e logica booleana, senza bisogno di semafori o meccanismi hardware.

42 Perché la Soluzione di Peterson Funziona

42.1 Introduzione

La Soluzione di Peterson garantisce correttamente mutua esclusione, progresso e attesa limitata grazie all'uso intelligente di due variabili condivise: `flag[]` e `turn`. Analizziamo il funzionamento e le ragioni del suo successo.

42.2 Variabili condivise

- `flag[i]`: indica l'intento del processo P_i di entrare nella sezione critica.
- `turn`: decide chi ha la precedenza in caso di conflitto.

42.3 Mutua esclusione

- Solo un processo può superare la condizione di attesa contemporaneamente.
- Spiegazione: se entrambi i processi impostano `flag[i] = true` e poi `turn = j`, solo il processo non indicato da `turn` può passare per primo.
- Questo impedisce che due processi entrino nella sezione critica simultaneamente.

42.4 Progresso

- Se l'altro processo non è interessato (`flag[j] = false`), la condizione di attesa diventa falsa.
- Spiegazione: il processo non deve attendere inutilmente; può entrare immediatamente nella sezione critica.
- Garantisce che la scelta di chi entra non venga ritardata indefinitamente.

42.5 Attesa limitata (Bounded Waiting)

- La variabile `turn` alterna la precedenza tra i processi.
- Una volta completata la sezione critica, il processo resetta `flag[i] = false`.
- Spiegazione: questo meccanismo impedisce che un processo venga rimandato indefinitamente, evitando la starvation.

42.6 Idea chiave

La Soluzione di Peterson funziona perché:

1. Coordina i processi tramite due semplici variabili condivise
2. Garantisce che solo un processo alla volta possa entrare nella sezione critica (**mutua esclusione**)
3. Permette ai processi di avanzare se l'altro non è interessato (**progresso**)
4. Alterna opportunamente la priorità e resetta i flag (**attesa limitata**)

In questo modo, la soluzione risolve elegantemente il problema della sezione critica per due processi, senza bisogno di semafori o supporto hardware speciale.

43 Mutex Locks

43.1 Introduzione

I **mutex** (mutual exclusion locks) sono strumenti software fondamentali per proteggere le **sezioni critiche** in sistemi con più processi o thread. Essi garantiscono che solo un processo alla volta possa accedere a una risorsa condivisa.

43.2 Funzionamento base

- Un processo deve **acquisire** il lock (`acquire()`) prima di entrare nella sezione critica.
- Dopo aver completato l'esecuzione della sezione critica, il processo **rilascia** il lock (`release()`).
- La variabile booleana `available` indica se il lock è libero:
  - `available = true` → il lock è disponibile
  - `available = false` → il lock è occupato
- Se il lock non è disponibile, il processo deve attendere finché non diventa libero.

43.3 Problema della Busy Waiting

- Nel metodo tradizionale, la funzione `acquire()` può entrare in un ciclo di **attesa attiva** (*busy waiting*) se il lock è occupato.
- Durante l'attesa attiva, il processo continua a controllare la disponibilità del lock, consumando cicli CPU inutilmente.
- Questo **spreco di risorse** è particolarmente problematico nei sistemi multiprogrammati, poiché riduce l'efficienza complessiva della CPU.

43.4 Schema di utilizzo di un mutex

```
// Processo P
acquire(mutex);   // entra nella sezione critica
// sezione critica
...
release(mutex);   // esce dalla sezione critica
// sezione restante
...
```

43.5 Idea chiave

I mutex forniscono un metodo semplice per garantire la **mutua esclusione** nelle sezioni critiche, ma l'implementazione deve evitare la **busy waiting** per non sprecare cicli CPU. Nelle implementazioni moderne si preferiscono tecniche basate su **sleep/wakeup** o semafori per ridurre l'overhead.

44 Semafori

44.1 Introduzione

I **semafori** sono strumenti fondamentali per la sincronizzazione dei processi nei sistemi operativi. Essi permettono di coordinare l'accesso a risorse condivise evitando **race conditions** e problemi di **busy waiting** presenti nei mutex tradizionali.

44.2 Definizione

Un semaforo S è una variabile intera accessibile esclusivamente tramite due operazioni atomiche:

- `wait(S)` (anche chiamata `P` o *proberen*)
- `signal(S)` (anche chiamata `V` o *verhogen*)

44.3 Proprietà delle operazioni

- Entrambe le operazioni devono essere **atomiche**: nessun altro processo può leggere o modificare il semaforo durante l'esecuzione di una di esse.
- **wait(S)**:
  - Controlla se $S \leq 0$
  - Se vero, il processo deve attendere
  - Se falso, decrementa S $(S--)$ senza interruzioni
- **signal(S)**:
  - Incrementa S $(S++)$ in modo atomico
  - Eventualmente sveglia uno dei processi in attesa

44.4 Importanza dell'atomicità

- L'atomicità garantisce che il valore del semaforo non venga corrotto da più processi che lo modificano simultaneamente.
- Evita la necessità di busy waiting, poiché i processi possono essere messi in *sleep* fino a quando il semaforo diventa disponibile.

44.5 Idea chiave

I semafori forniscono un meccanismo robusto per la sincronizzazione dei processi e la protezione delle sezioni critiche, garantendo:

1. Accesso sicuro alle risorse condivise
2. Assenza di race conditions
3. Eliminazione o riduzione della busy waiting

44.6 Schema di utilizzo di un semaforo

```
// Ingresso in sezione critica
wait(S);
// sezione critica
...
// Uscita dalla sezione critica
signal(S);
```

45 Semafori Contatori e Binari

45.1 Introduzione

I semafori possono essere classificati in due tipi principali, a seconda della gamma di valori che possono assumere:

- **Semafori binari** (*binary semaphores*)
- **Semafori contatori** (*counting semaphores*)

45.2 Semafori binari

- Possono assumere solo due valori: 0 o 1.
- Funzionano in modo simile ai **mutex**, permettendo a un solo processo di accedere alla sezione critica alla volta.
- Esempio: un semaforo binario inizializzato a 1 indica che la risorsa è libera; se un processo esegue `wait()`, il semaforo diventa 0 e gli altri processi devono attendere.

45.3 Semafori contatori

- Possono assumere qualsiasi valore intero maggiore o uguale a zero.
- Permettono la gestione di risorse con **più unità disponibili**.
- Funzionamento:
  - `wait(S)` decrementa il contatore
  - `signal(S)` incrementa il contatore
  - Quando il contatore è 0, tutte le unità della risorsa sono occupate e i processi devono attendere finché non si libera almeno un'unità.
- Esempio: un semaforo contatore inizializzato a N gestisce N unità identiche di una risorsa condivisa.

Differenze principali

Caratteristica	Semaforo Binario	Semaforo Contatore
Valori possibili	0 o 1	Interi ≥ 0
Uso tipico	Protezione sezione critica singola	Gestione risorse multiple
Esempio	Mutex	Controllo accesso a N stampanti

Tabella 1: Confronto tra semafori binari e contatori

45.4 Idea chiave

I semafori binari e contatori rappresentano strumenti flessibili per la sincronizzazione:

- **Binario**: garantisce mutua esclusione su una risorsa singola
- **Contatore**: consente l'accesso controllato a più unità di una risorsa, evitando congestione e race conditions.

46 Semafori per la Sincronizzazione dei Processi

46.1 Introduzione

Oltre a proteggere le sezioni critiche, i semafori sono strumenti potenti per **sincronizzare l'ordine di esecuzione** tra processi. Un caso tipico è garantire che un'operazione S_2 venga eseguita solo dopo il completamento di un'altra operazione S_1.

46.2 Meccanismo base

- Si utilizza un semaforo condiviso inizializzato a 0:

```
sem synch = 0;
```

- Processo P_1 esegue S_1 e poi segnala il semaforo con `signal(synch)`, indicando che S_1 è completato.
- Processo P_2 esegue `wait(synch)` prima di S_2. Se P_1 non ha ancora completato S_1, P_2 viene sospeso fino al segnale.

46.3 Codice in pseudolinguaggio

```
// Processo P1
S1;                 // esegue l'operazione S1
signal(synch);      // segnala a P2 che può procedere

// Processo P2
wait(synch);        // attende che P1 completi S1
S2;                 // esegue l'operazione S2
```

46.4 Spiegazione dettagliata

- Il semaforo `synch` agisce come un meccanismo di **barriera**, bloccando P_2 finché P_1 non ha terminato S_1.
- La chiamata `wait(synch)` decrementa il semaforo; se il valore è negativo, il processo viene sospeso.
- La chiamata `signal(synch)` incrementa il semaforo e sveglia eventualmente un processo sospeso.
- Questo schema garantisce la sincronizzazione senza busy waiting, evitando sprechi di CPU.

46.5 Idea chiave

I semafori non servono solo per proteggere sezioni critiche, ma anche per **coordinare l'ordine di esecuzione dei processi**, rendendoli strumenti fondamentali nella programmazione concorrente.

47 Evitare la Busy Waiting con i Semafori

47.1 Introduzione

Nei semafori tradizionali, un processo che esegue `wait()` su un semaforo occupato può entrare in **busy waiting**, consumando cicli CPU inutilmente. Nei sistemi moderni, questo problema si evita **bloccando il processo** e mettendolo in attesa, consentendo al CPU scheduler di eseguire altri processi pronti.

47.2 Blocco del processo

- Quando un processo esegue `wait(S)` e trova $S \leq 0$, non esegue più cicli di attesa attiva.
- Il processo viene **bloccato** e inserito in una **coda di attesa** associata al semaforo S.
- Il controllo passa al **scheduler**, che seleziona un altro processo pronto da eseguire.

47.3 signal() e wakeup()

- Quando un processo esegue `signal(S)`, il semaforo viene incrementato.
- Se vi sono processi bloccati in attesa, `signal()` **risveglia** uno di essi.
- Il processo bloccato cambia stato da **waiting** a **ready** e viene reinserito nella coda dei processi pronti.
- Il scheduler, a questo punto, potrà scegliere il processo risvegliato per l'esecuzione.

47.4 Schema concettuale

```
// wait() modificato senza busy waiting
wait(S):
    S--;
    if S < 0:
        add process to S.waiting_queue
        block process
        scheduler runs another ready process

// signal() modificato
signal(S):
    S++;
    if S.waiting_queue not empty:
        remove a process from waiting_queue
        wakeup(process)
```

47.5 Idea chiave

Questo approccio elimina la **busy waiting**, aumentando l'efficienza della CPU. I processi non consumano cicli CPU mentre attendono la disponibilità della risorsa; vengono risvegliati solo quando il semaforo segnala che la risorsa è disponibile. Così, semafori e scheduler lavorano insieme per garantire sincronizzazione e ottimizzazione delle risorse.

48 Struttura e Operazioni dei Semafori

48.1 Struttura di un Semaforo

Un semaforo è una struttura dati fondamentale per la sincronizzazione dei processi. Contiene due componenti principali:

- **value**: un intero che rappresenta il numero di risorse disponibili o il permesso per un processo di continuare.
- **list**: una coda di processi che devono attendere quando `value` diventa negativo.

48.2 Operazione wait()

L'operazione `wait(S)` (nota anche come `P` o *proberen*) ha il compito di decrementare il semaforo e bloccare il processo se necessario:

1. Decrementa `S->value` di 1.
2. Se `S->value < 0`:
   - Il processo corrente viene aggiunto alla **coda di attesa** del semaforo.
   - Il processo viene **bloccato** fino a quando non viene risvegliato da una chiamata `signal()`.
3. Se `S->value  0`, il processo può continuare immediatamente senza blocchi.

48.3 Operazione signal()

L'operazione `signal(S)` (nota anche come `V` o *verhogen*) ha il compito di incrementare il semaforo e risvegliare eventuali processi in attesa:

1. Incrementa `S->value` di 1.
2. Se `S->value  0`:
   - Un processo viene rimosso dalla **coda di attesa**.
   - Il processo viene **risvegliato** e inserito nella coda dei processi pronti.
3. Se `S->value > 0`, nessun processo era in attesa, quindi non succede nulla.

48.4 Schema concettuale

```
// wait(S)
S->value--;
if (S->value < 0) {
    add process to S->list;
    block process;
} else {
    continue execution;
}

// signal(S)
S->value++;
if (S->value <= 0) {
    remove process from S->list;
    wakeup(process);  // process becomes ready
} else {
    // nothing to do
}
```

48.5 Idea chiave

Questa struttura garantisce:

- La gestione sicura delle risorse condivise.
- La sincronizzazione dei processi senza busy waiting.
- La possibilità di risvegliare i processi solo quando una risorsa diventa disponibile.

49 Comportamento della Coda dei Semafori

49.1 Introduzione

I semafori non solo contano le risorse disponibili, ma gestiscono anche i processi in attesa. La coda associata al semaforo determina l'ordine in cui i processi bloccati vengono risvegliati.

49.2 Valore del semaforo

- Il valore di un semaforo può diventare **negativo**.
- L'**assoluto** del valore del semaforo indica il numero di processi attualmente in attesa.
- Esempio: se $S = -3$, significa che tre processi sono bloccati sulla coda di questo semaforo.

49.3 Struttura del semaforo

Ogni semaforo contiene:

- Un **intero** che rappresenta il valore corrente (`S->value`).
- Un **puntatore a una coda di PCB** (*Process Control Block*) dei processi in attesa.

49.4 Politica della coda

- Una coda **FIFO** (First-In-First-Out) garantisce **bounded waiting**, evitando starvation.
- Tuttavia, è possibile utilizzare qualsiasi altra politica di coda a seconda delle esigenze del sistema operativo.

49.5 Funzionamento

- Quando un processo esegue `wait(S)` e il valore diventa negativo, il processo viene aggiunto alla coda.
- Quando un processo esegue `signal(S)` e ci sono processi in coda, uno di essi viene rimosso e risvegliato.
- L'ordine di risveglio dipende dalla politica della coda scelta.

49.6 Idea chiave

La coda dei semafori:

- Mantiene traccia dei processi in attesa in modo ordinato.
- Permette di implementare politiche di scheduling flessibili.
- Assicura, se FIFO è utilizzata, che l'attesa sia limitata e nessun processo subisca starvation indefinita.

50 Limiti dei Semafori

50.1 Introduzione

I semafori sono strumenti potenti per la sincronizzazione dei processi, ma il loro uso scorretto può portare a errori rari e difficili da individuare. La protezione delle sezioni critiche richiede attenzione estrema nell'ordine delle operazioni `wait()` e `signal()`.

50.2 Problemi comuni

- L'uso improprio può permettere a più processi di entrare contemporaneamente nella sezione critica, violando la **mutua esclusione**.
- Gli errori possono derivare da:
  - **Errori di programmazione**: dimenticanze, inversioni di ordine tra `wait()` e `signal()`, condizioni di race.
  - **Mancanza di cooperazione tra processi**: ad esempio, un processo che non esegue `signal()` dopo aver completato la sezione critica.
- Questi errori sono spesso **difficili da rilevare** perché si manifestano solo in particolari sequenze di esecuzione e concorrenza.

50.3 Esempio di errore

Supponiamo di avere un semaforo binario `mutex` per proteggere una sezione critica:

```
wait(mutex);
// sezione critica
signal(mutex);
```

- Se il `signal(mutex)` viene dimenticato, altri processi rimangono bloccati indefinitamente.
- Se l'ordine è invertito (`signal(mutex)` prima di `wait(mutex)`), due processi possono entrare contemporaneamente nella sezione critica.

50.4 Idea chiave

I semafori richiedono disciplina rigorosa nell'uso:

- Ogni `wait()` deve corrispondere a un `signal()`.
- La sequenza deve essere eseguita in maniera coerente per tutti i processi.
- La cooperazione dei processi è fondamentale per garantire sicurezza e correttezza.

Anche se potenti, i semafori non prevengono automaticamente errori logici o di programmazione: la correttezza dipende dall'implementazione e dall'uso disciplinato.

51 Monitor

51.1 Introduzione

I **monitor** sono strumenti di sincronizzazione integrati in alcuni linguaggi di programmazione moderni. Essi combinano il concetto di **tipo astratto di dato (ADT)** con meccanismi di sincronizzazione, permettendo l'accesso sicuro ai dati condivisi senza la gestione manuale di semafori o mutex.

51.2 Caratteristiche principali

- Solo **un processo alla volta** può eseguire codice all'interno di un monitor, garantendo mutua esclusione.
- I dati condivisi sono protetti: le variabili locali del monitor sono accessibili solo dalle procedure interne al monitor.
- Le procedure del monitor operano sui dati interni e sui parametri ricevuti, rispettando l'incapsulamento.
- Ogni monitor include:
  - Variabili interne (**stato interno**)
  - Operazioni che manipolano lo stato

51.3 Encapsulamento e sicurezza

- Il **tipo del monitor** non può essere utilizzato direttamente dai processi esterni.
- Le funzioni interne possono accedere solo alle variabili locali del monitor e ai loro parametri.
- Questo garantisce:
  - **Encapsulamento rigoroso**
  - **Accesso sicuro** ai dati condivisi
  - Eliminazione della sincronizzazione manuale e del rischio di errori da uso improprio di semafori

51.4 Schema di Sintassi Tipica di un Monitor

Un monitor può essere definito come segue (pseudocodice generico):

```
// Definizione del monitor
monitor NomeMonitor {
    // Variabili condivise (stato interno)
    var sharedData1;
    var sharedData2;

    // Procedure (operazioni protette)
    procedure operazione1(parametri) {
        // codice che accede e modifica sharedData1/2
    }

    procedure operazione2(parametri) {
        // codice che accede e modifica sharedData1/2
    }

    // Condizioni per la sincronizzazione
    condition condizione1;
    condition condizione2;
}
```

51.5 Note sul funzionamento

- Solo un processo alla volta può eseguire qualsiasi procedura del monitor.
- Le `condition` servono per coordinare i processi all'interno del monitor (tramite operazioni tipo `wait(condizione)` e `signal(condizione)`).
- Il monitor gestisce automaticamente la mutua esclusione: non è necessario usare semafori o mutex esterni.

51.6 Idea chiave

I monitor rappresentano un livello superiore di astrazione rispetto a semafori e mutex:

- Forniscono sincronizzazione automatica.
- Permettono di scrivere codice concorrente più chiaro e sicuro.
- Mantengono la sicurezza dei dati condivisi tramite incapsulamento e mutua esclusione automatica.

52 Variabili di Condizione nei Monitor

52.1 Introduzione

Nei monitor, la mutua esclusione è garantita automaticamente: solo un processo alla volta può eseguire codice all'interno del monitor. Tuttavia, alcuni schemi di sincronizzazione richiedono un ulteriore meccanismo: le **variabili di condizione**.

52.2 Definizione

- Le variabili di condizione sono definite all'interno del monitor, ad esempio:

```
condition x, y;
```

- Esse servono a sospendere e risvegliare processi basandosi su condizioni logiche specifiche.
- Ogni variabile di condizione supporta solo due operazioni:
  - `wait()`: sospende il processo chiamante fino a quando un altro processo esegue `signal()` sulla stessa variabile.
  - `signal()`: risveglia esattamente un processo sospeso su quella variabile.

52.3 Funzionamento di wait() e signal()

- `x.wait()`:
  - Il processo che chiama `wait()` rilascia temporaneamente il controllo del monitor.
  - Il processo viene sospeso e inserito nella coda della variabile di condizione `x`.
  - Il monitor permette a un altro processo pronto di entrare ed eseguire codice.
- `x.signal()`:
  - Risveglia esattamente un processo sospeso nella coda della variabile di condizione `x`.
  - Il processo risvegliato viene inserito nella coda dei processi pronti del monitor e attende di riprendere l'esecuzione.

52.4 Idea chiave

Le variabili di condizione permettono di sincronizzare processi all'interno del monitor:

- Gestiscono **dipendenze logiche** tra processi (ad esempio Produttore-Consumatore).
- Combinandosi con la mutua esclusione automatica del monitor, eliminano la necessità di semafori manuali.
- Garantendo che solo i processi realmente pronti vengano risvegliati, evitano busy waiting e migliorano l'efficienza.

53 Comportamento di `signal()` con le Variabili di Condizione

53.1 Introduzione

Le variabili di condizione permettono di sincronizzare i processi all'interno di un monitor. Il comportamento dell'operazione `signal()` su una variabile di condizione è diverso da quello dei semafori e deve essere compreso con attenzione.

53.2 Comportamento base di signal()

- Se **nessun processo è sospeso** sulla variabile di condizione, `signal()` non ha alcun effetto.
- Questo differisce dal `signal()` dei semafori, che incrementa sempre lo stato del semaforo anche se nessun processo è in attesa.

53.3 Caso con processi sospesi

- Se un processo P esegue `x.signal()` e esiste almeno un processo sospeso Q sulla variabile `x`, il processo Q viene risvegliato.
- Solo **uno dei due processi** può eseguire all'interno del monitor alla volta, poiché la mutua esclusione è garantita automaticamente.

53.4 Politiche di risveglio

Esistono due possibili politiche quando un processo viene risvegliato:

1. **Signal-and-wait:** Il processo P che esegue `signal()` cede immediatamente il monitor e sospende la propria esecuzione fino a quando Q lascia il monitor o entra in attesa su un'altra variabile. *Vantaggio:* Q entra subito nella sezione critica; *Svantaggio:* può introdurre maggiore contesto switch se P deve riprendere subito dopo.
2. **Signal-and-continue:** Il processo P continua a eseguire nel monitor mentre Q attende che P lasci il monitor o entri in attesa su un'altra variabile. *Vantaggio:* riduce il numero di context switch; *Svantaggio:* il processo risvegliato Q deve aspettare più a lungo per accedere al monitor.

53.5 Idea chiave

- Entrambe le politiche sono valide e rispettano la mutua esclusione del monitor.
- La scelta dipende dal compromesso tra **tempestività del risveglio** e **efficienza del contesto di esecuzione**.
- È fondamentale comprendere questa differenza quando si progettano algoritmi di sincronizzazione avanzati con monitor.

54 Implementazione dei Monitor tramite Semafori

54.1 Introduzione

I monitor possono essere implementati utilizzando semafori come strumenti sottostanti per garantire la mutua esclusione e la gestione dei processi sospesi sulle variabili di condizione. Questa implementazione è particolarmente utile per comprendere il funzionamento interno dei monitor.

54.2 Semafori utilizzati

- **mutex** (`mutex = 1`): Garantisce la mutua esclusione, consentendo che solo un processo alla volta esegua all'interno del monitor.
- **next** (`next = 0`): Gestisce i processi che vengono sospesi da un `signal()` seguendo la politica **signal-and-wait**.
- **next_count**: Contatore dei processi sospesi sul semaforo `next`.

54.3 Funzionamento della politica signal-and-wait

1. Quando un processo esegue `signal()` su una variabile di condizione:
   - Se ci sono processi in attesa, il processo corrente si sospende su `next`.
   - Il semaforo `next` viene utilizzato per trasferire immediatamente il controllo al processo risvegliato, senza rilasciare `mutex`.
2. L'ultimo processo attivo all'interno del monitor:
   - Esegue `signal(next)` se ci sono altri processi sospesi su `next`.
   - Altrimenti esegue `signal(mutex)` per rilasciare il monitor.

54.4 Schema concettuale di implementazione

```
// Ingresso nel monitor
wait(mutex); // acquisizione mutua esclusione

// Esecuzione di una procedura del monitor
if (condizione_da_wait) {
    wait(condizione); // sospende processo sulla variabile di condizione
}

// Segnale di una variabile di condizione
signal(condizione) {
    if (processi_in_attesa) {
        next_count++;
        signal(next); // trasferisce il controllo al processo risvegliato
        wait(next);   // sospende il processo corrente
        next_count--;
    }
}

// Uscita dal monitor
if (next_count > 0) {
    signal(next);
} else {
    signal(mutex); // rilascia il monitor
}
```

54.5 Idea chiave

- La mutua esclusione è garantita dal semaforo `mutex`.
- La politica **signal-and-wait** assicura che un processo risvegliato entri subito nel monitor, mantenendo la correttezza della sincronizzazione.
- L'uso dei semafori `next` e `next_count` evita race condition tra processi che eseguono `signal()`.
- Questo modello mostra come i concetti teorici dei monitor possono essere realizzati in sistemi operativi che forniscono solo primitive di basso livello (semafori).

55 Implementazione delle Variabili di Condizione tramite Semafori

55.1 Introduzione

Le variabili di condizione nei monitor possono essere implementate utilizzando semafori per sospendere e risvegliare i processi in maniera controllata. Ogni variabile di condizione gestisce i processi in attesa tramite un semaforo dedicato e un contatore. Questo approccio garantisce la corretta sincronizzazione rispettando la mutua esclusione del monitor.

55.2 Struttura per ogni variabile di condizione

Per ogni variabile di condizione x:

- Un semaforo `x_sem` inizializzato a 0, che serve per sospendere e risvegliare i processi.
- Un contatore `x_count` inizializzato a 0, che tiene traccia del numero di processi sospesi sulla variabile x.

55.3 Funzionamento di `x.wait()`

Quando un processo chiama `x.wait()`:

1. Incrementa `x_count`, indicando che un processo si sospenderà sulla variabile di condizione.
2. Lascia temporaneamente il monitor:
   - Se ci sono processi sospesi sul semaforo `next`, viene eseguito `signal(next)` per trasferire il controllo.
   - Altrimenti, rilascia il semaforo `mutex` del monitor.
3. Blocca il processo sul semaforo `x_sem`, sospendendo la sua esecuzione fino a quando un altro processo esegue `x.signal()`.
4. Quando il processo viene risvegliato:
   - Decrementa `x_count`.
   - Riprende l'esecuzione all'interno del monitor.

55.4 Funzionamento di `x.signal()`

Quando un processo chiama `x.signal()`:

- Ha effetto solo se `x_count > 0`, cioè ci sono processi sospesi sulla variabile di condizione.
- Risveglia un processo sospeso eseguendo `signal(x_sem)`.
- Il processo chiamante sospende la propria esecuzione su `next` fino a quando il processo risvegliato completa la sua esecuzione nel monitor o esegue un altro `wait()`.

55.5 Schema concettuale di pseudocodice

```
// wait() per la variabile di condizione x
x.wait() {
    x_count++;
    if (next_count > 0)
        signal(next);
    else
        signal(mutex);
    wait(x_sem); // sospensione del processo
    x_count--;
}

// signal() per la variabile di condizione x
x.signal() {
    if (x_count > 0) {
        next_count++;
        signal(x_sem);  // risveglia un processo sospeso su x
        wait(next);     // sospende il processo corrente
        next_count--;
    }
}
```

55.6 Idea chiave

- Questa implementazione garantisce che solo un processo alla volta esegua nel monitor.
- I processi sospesi su una variabile di condizione attendono in modo ordinato sul semaforo dedicato.
- La politica **signal-and-wait** assicura che un processo risvegliato entri immediatamente nel monitor senza violare la mutua esclusione.
- L'uso di contatori e semafori separati evita race condition tra processi che chiamano `signal()` simultaneamente.

56 Ripresa dei Processi all'interno di un Monitor

56.1 Introduzione

Quando più processi sono sospesi su una variabile di condizione, la chiamata `x.signal()` deve decidere quale processo risvegliare per primo. La scelta del processo da risvegliare influisce sull'efficienza, sulla prevedibilità e sulla correttezza degli algoritmi concorrenti.

56.2 Strategie di selezione del processo

- **FCFS (First-Come, First-Served)**: Il processo che ha chiamato `x.wait()` per primo viene risvegliato per primo. *Vantaggio: semplice da implementare; svantaggio: non sempre ottimale per la priorità dei processi.*
- **Conditional-wait con priorità:** Si estende `x.wait()` aggiungendo un parametro di priorità c:

  ```
  x.wait(c); // c = valore di priorità del processo
  ```

  In questo caso:

  - Ogni processo sospeso mantiene un valore di priorità.
  - Alla chiamata `x.signal()`, il monitor seleziona il processo con **il valore di priorità più basso** per riprendere l'esecuzione.
  - Questo meccanismo permette di risolvere conflitti tra processi con esigenze diverse, migliorando la reattività dei processi critici.

56.3 Esempio concettuale

Supponiamo tre processi P_1, P_2, P_3 sospesi su `x` con priorità rispettivamente 10, 5 e 7.

- Alla chiamata `x.signal()`, il monitor risveglierà P_2 (priorità più bassa = 5).
- Successivamente, chiamate successive a `x.signal()` risveglieranno P_3 e infine P_1.

56.4 Idea chiave

- La gestione intelligente della ripresa dei processi all'interno del monitor migliora la **determinismo** e la **performance** dei sistemi concorrenti.
- L'uso della priorità consente di adattare la sincronizzazione alle esigenze dei processi, evitando che processi critici siano ritardati da altri meno urgenti.
- La strategia FCFS è semplice, ma per applicazioni real-time o con processi di importanza diversa, la priorità è più adatta.